

A
Project Report on
**MACHINE LEARNING-BASED CLASSIFICATION OF SLEEP
PATTERNS AND DISORDERS**

Submitted in partial fulfillment of the requirements
for the award of the degree of
BACHELOR OF TECHNOLOGY

In
ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Submitted by

M.RAMANA KUMAR REDDY	21AK1A3063
P.PAVAN KUMAR REDDY	21AK1A3054
B.PUNEETH	21AK1A3060
K.VENKATA SUDEEP	21AK1A3093

Under the guidance of

Ms. V S Gayathri., M.Tech
Assistant Professor



ARTIFICIAL INTELLIGENCE
ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES
(AUTONOMOUS)

(Approved by AICTE, New Delhi & Permanent Affiliation to JNTUA, Anantapur.
Three B. Tech Programmes (CSE, ECE & CE) are accredited by NBA, New Delhi, Accredited by NAAC
with 'A' Grade, Bangalore. Accredited by Institution of Engineers (India), KOLKATA. A-grade
awarded by AP Knowledge Mission. Recognized under sections 2(f) & 12(B) of UGC Act 1956.)

Tirupati-517520.

2021-2025

**ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES
(AUTONOMOUS)**

(Approved by AICTE, New Delhi & Permanent Affiliation to JNTUA, Anantapur.
Three B. Tech Programmes (CSE, ECE & CE) are accredited by NBA, New Delhi, Accredited by NAAC
with 'A' Grade, Bangalore. Accredited by Institution of Engineers (India), KOLKATA. A-grade
awarded by AP Knowledge Mission. Recognized under sections 2(f) & 12(B) of UGC Act 1956.)

Tirupati-517520.

2021-2025

ARTIFICIAL INTELLIGENCE



CERTIFICATE

*Certified that this is a bonafide record of the Project Report entitled, **"MACHINE LEARNING-BASED CLASSIFICATION OF SLEEP PATTERNS AND DISORDERS"**, done by **M Ramana Kumar Reddy (21AK1A3063)**, **P Pavan Kumar Reddy (21AK1A3054)**, **B Puneeth (21AK1A3060)**, **K Venkata Sudeep (21AK1A3093)** is being submitted in partial fulfilment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in **ARTIFICIAL INTELLIGENCE AND DATA SCIENCE** to the Annamacharya Institute of technology and Sciences, Tirupati, during the academic year 2021-25.*

Signature of the supervisor

Ms. V S Gayathri, M.Tech

Assistant professor,

Department of AI,

AITS, Tirupati.

Signature of the Co-Guide

Dr. P. Krishnajyothi, M.Sc, Ph.D

Assistant professor,

Department of H & S,

AITS, Tirupati.

Signature of HOD

Dr. C. Siva Balaji Yadav.,

M.Tech, Ph.D

Professor & HOD,

Department of AI,

AITS, Tirupati.

DATE:

INTERNAL EXAMINER

EXTERNAL EXAMINER

ANNAMACHARYA INSTITUTE OF TECHNOLOGY AND SCIENCES (AUTONOMOUS)

(Approved by AICTE, New Delhi & Permanent Affiliation to JNTUA, Anantapur.
Three B. Tech Programmes (CSE, ECE & CE) are accredited by NBA, New Delhi, Accredited by NAAC
with 'A' Grade, Bangalore. Accredited by Institution of Engineers (India), KOLKATA. A-grade
awarded by AP Knowledge Mission. Recognized under sections 2(f) & 12(B) of UGC Act 1956.)
Tirupati-517520.
2021-2025

ARTIFICIAL INTELLIGENCE



DECLARATION

We hereby declare that the project titled **“Machine Learning-Based Classification of Sleep Patterns and Disorders ”** is a genuine project work carried out by us, in B.Tech (Artificial Intelligence & Data Science) course in Annamacharya Institute of Technology And Sciences and has not been submitted to any other course or university for the award of our degree by us.

M Ramana Kumar Reddy
P Pavan Kumar Reddy
B Puneeth
K Venkata Sudeep

21AK1A3063
21AK1A3054
21AK1A3060
21AK1A3093

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of the task would be put incomplete without the mention of the people who made it possible, whose constant guidance and encouragement crown all the efforts with success.

We avail this opportunity to express our deep sense of gratitude and hearty thanks to **Dr. C. GANGI REDDY**, Hon'ble Secretary of AITS-Tirupati, for providing congenial atmosphere and encouragement.

We show gratitude to **Dr. C. NADHAMUNI REDDY, Principal** for having provided all the facilities and support.

We would like to thank **Dr. C. Siva Balaji Yadav, M.Tech, Ph.D Professor & HOD, Artificial Intelligence and Data Science** for encouragement at various levels of our Project.

We thankful to our project coordinator **Mr. E D Pavankumar, M.Tech, (Ph.D), Assistant Professor, AI**, for his sustained inspiring guidance and cooperation throughout the process of this project.

We would like to express our sincere gratitude to our supervisor **Ms. V S Gayathri, M.Tech, Assistant Professor**, Dept. of AI, AITS, for her constant help, kind cooperation and encouragement in completing the work successfully.

We would like to express our sincere gratitude to our Co-Guide **Ms.P Krishnajyothi, M.Sc, Ph.D, Assistant Professor**, Dept. of H&S, AITS, for her constant help, kind cooperation and encouragement in completing the work successfully.

We express our deep sense of gratitude and thanks to all the **Teaching and Non-Teaching Staff** of our college who stood with us during the project and helped us to make it a successful venture.

We place highest regards to our **Parents, Friends and Well wishers** who helped a lot in making the report of this project.

M Ramana Kumar Reddy
P Pavan Kumar Reddy
B Puneeth
K Venkata Sudeep

21AK1A3063
21AK1A3054
21AK1A3060
21AK1A3093

CONTENTS

CHAPTER NO	NAME OF THE CHAPTER	PAGE NO
	Abstract	i
	List of figures	ii
	List of screens	iii
	List of abbreviations	iv
Chapter 1	INTRODUCTION	
1.1	INTRODUCTION	1
1.2	EXISTING SYSTEM	3
1.3	DISADVANTAGES OF EXISTING SYSTEM	3
1.4	PROPOSED SYSTEM	4
1.5	ADVANTAGES OF PROPOSED SYSTEM	4
Chapter 2	ANALYSIS	
2.1	INTRODUCTION	5
2.2	SOFTWARE REQUIREMENTS SPECIFICATION	5
2.2.1	User Requirements	5
2.2.2	Software Requirements	6
2.2.3	Hardware Requirements	7
2.3	FLOWCHART	7
Chapter 3	DESIGN	
3.1	STEPS INVOLVED IN PHISHING DETECTION	9
3.1.1	Data Gathering and Data Pre-processing	9
3.1.2	Training the Model	9
3.1.3	Final Prediction model integration	10
3.2	MODULES INVOLVED IN PHISHING DETECTION	10
3.3	DFD/UML DIAGRAMS	14
Chapter 4	IMPLEMENTATION & DETAILS	
4.1	INTRODUCTION	23
4.2	EXPLANATION OF KEY FUNCTIONS	35
4.3	METHOD OF IMPLEMENTATION	36
4.3.1	Machine Learning Models	36
4.3.2	Output Screens	41

4.3.3	Result Analysis	44
Chapter 5	TESTING & VALIDATION	
5.1	INTRODUCTION	45
5.2	DESIGN OF EVALUATION METRICS	50
5.3	VALIDATION	50
Chapter 6	CONCLUSION & FUTURE ENHANCEMENT	
6.1	CONCLUSION	51
6.2	FUTURE ENHANCEMENT	52
Chapter 7	BIBILOGRAPHY	53

APPENDIX

S.NO	PUBLICATIONS
1.	National Conference Certificates
2.	International Journal Paper
3.	International Journal Certificates

ABSTRACT

Sleep disorders significantly impact physical and mental health, necessitating accurate and accessible diagnostic methods. Traditional diagnostic techniques like Polysomnography (PSG) are often inconvenient, expensive, and limited in availability. This project aims to leverage machine learning algorithms to classify sleep disorders using health and lifestyle data from the Kaggle Sleep Health and Lifestyle Dataset. The existing system employs algorithms such as K-Nearest Neighbors (KNN), Support Vector Machine (SVM), Decision Tree, Random Forest, and Artificial Neural Network (ANN), which have several limitations including computational expense and sensitivity to hyperparameters. To address these issues, the proposed system implements ensemble learning techniques, specifically Stacking Classifier and Voting Classifier, to enhance accuracy, robustness, and interpretability. By combining the strengths of multiple models, the project seeks to provide a more efficient, cost-effective, and accessible solution for diagnosing sleep disorders, ultimately improving patient outcomes and quality of life.

Keywords: Sleep Disorders, Machine Learning, Stacking Classifier, Voting Classifier, Sleep Apnea, Insomnia.

LIST OF FIGURES

Fig No	Name	Page No
2.1	Flowchart for Phishing Detection	8
3.1	Level-0 DFD	14
3.2	Level-1 DFD	14
3.3	Class Diagram	15
3.4	Use Case Diagram	16
3.5	Sequence Diagram	17
3.6	Deployment Diagram	18
3.7	Activity Diagram	19
3.8	Collabiration Diagram	20
3.9	Component Diagram	21
3.10	ER Model	21
3.11	Content Diagram	22

LIST OF SCREENS

Screen No	Name	Page No
4.1	Dataset Loading	41
4.2	Columns in dataset Screen	41
4.3	Support Vector Machine Model	42
4.4	K-Nearest Neighbors Model	42
4.5	Decision Tree Model	42
4.6	Random Forest	43
4.7	Artificial Neural Network	43
4.8	Stacking Classifier	43
4.9	Voting Classifier	44
4.10	Result Analysis	44

LIST OF ABBREVIATIONS

Short Form	Abbreviation
ML	Machine Learning
SVM	Support Vector Machine
RF	Random Forest
DT	Decision Trees
KNN	K-Nearest Neighbours
PNG	Polysomnography
ANN	Artificial Neural Network
VC	Voting Classifier
SC	Stacking Classifier

1. INTRODUCTION

1.1 INTRODUCTION

The purpose of this project is to accurately classify individuals as having a sleep disorder or not, using advanced machine learning algorithms to improve accessibility and diagnostic efficiency.

Sleep disorders affect millions globally, posing significant health risks and impacting quality of life. Traditional diagnostic methods like Polysomnography (PSG) are costly, inconvenient, and limited in accessibility, requiring overnight stays in specialized centers. These methods also involve extensive preparation and follow-up, adding to patient burden. There is a critical need for an efficient, cost-effective, and accessible diagnostic system. This project aims to address this need by developing a machine learning-based classification system using health and lifestyle data to accurately identify sleep disorders, providing a more practical and widely available diagnostic solution.

The motivation for this project stems from the growing prevalence of sleep disorders and the limitations of traditional diagnostic methods like Polysomnography (PSG), which are inconvenient, expensive, and not widely accessible. Many individuals suffer from undiagnosed sleep disorders due to these barriers. By utilizing advanced machine learning algorithms and ensemble techniques, we aim to develop a more accessible, cost-effective, and efficient diagnostic tool. This project aspires to improve early detection and treatment of sleep disorders, enhancing patient outcomes and overall quality of life, and contributing to the broader field of healthcare innovation.

The scope of this project includes developing a machine learning-based system to classify sleep disorders using health and lifestyle data. The system will utilize both traditional algorithms (KNN, SVM, Decision Tree, Random Forest, ANN) and advanced ensemble learning techniques (Stacking Classifier, Voting Classifier) to enhance diagnostic accuracy and robustness. The project involves data collection, preprocessing, model training, evaluation, and optimization. It aims to provide a more accessible and cost-effective diagnostic tool compared to traditional methods, ultimately improving the detection and management of sleep disorders for a broader population. Future work may explore incorporating deep learning models and real-time applications.

Sleep disorders are prevalent health issues that significantly affect millions of individuals worldwide, leading to adverse effects on physical and mental health. Disorders

such as insomnia, sleep apnea, and restless legs syndrome can cause chronic fatigue, reduced cognitive function, and increased risk of various medical conditions. Accurate diagnosis and classification of sleep disorders are crucial for effective treatment and management. However, traditional diagnostic methods, such as Polysomnography (PSG), present several challenges that limit their accessibility and efficiency.

Polysomnography, considered the gold standard for diagnosing sleep disorders, requires patients to stay overnight in specialized sleep laboratories. This procedure involves significant preparation, follow-up visits, and the use of costly equipment and specialized healthcare personnel. Additionally, PSG is often limited to specialized centers, making it inaccessible to individuals in remote or underserved areas. The high costs associated with PSG and long waiting times further complicate timely diagnosis and treatment, often leading to delays in patient care.

To address these limitations, this project aims to apply machine learning algorithms for the classification of sleep disorders using health and lifestyle data. By leveraging advanced machine learning techniques, we can develop a more accessible, cost-effective, and efficient system for diagnosing sleep disorders. The dataset used in this project is sourced from Kaggle and includes various health and lifestyle factors that influence sleep patterns, providing a comprehensive foundation for machine learning models.

The existing system employs traditional machine learning algorithms such as K-Nearest Neighbors (KNN), Support Vector Machine (SVM), Decision Tree, Random Forest, and Artificial Neural Network (ANN). These algorithms have been widely used due to their simplicity and effectiveness in handling various classification tasks. However, they come with several disadvantages, including computational expense, sensitivity to hyperparameters, risk of overfitting, and lack of interpretability.

To overcome these challenges, the proposed system implements ensemble learning techniques, specifically the Stacking Classifier and Voting Classifier. Ensemble learning methods combine the strengths of multiple base models to enhance overall performance and robustness. The Stacking Classifier involves training several base models and then using another model to combine their predictions. This method leverages the individual strengths of different algorithms to achieve better predictive performance. The Voting Classifier aggregates the predictions of multiple models through majority voting or averaging, providing a more stable and accurate classification outcome.

The primary objective of this project is to develop a machine learning-based system that accurately classifies whether an individual has a sleep disorder or not, based on health and

lifestyle data. By implementing advanced ensemble learning techniques, the proposed system aims to provide a more reliable and interpretable solution, addressing the limitations of traditional diagnostic methods and enhancing accessibility for a broader population. The scope of this project includes data collection, preprocessing, model training, evaluation, and optimization, with future work potentially exploring the incorporation of deep learning models and real-time applications.

Ultimately, this project seeks to improve the detection and management of sleep disorders, contributing to better patient outcomes and quality of life through innovative and accessible machine learning solutions.

1.2 Existing System

The existing system for the classification of sleep disorders relies on traditional machine learning algorithms such as K-Nearest Neighbors (KNN), Support Vector Machine (SVM), Decision Tree, Random Forest, and Artificial Neural Network (ANN). These algorithms are commonly used due to their simplicity and effectiveness in handling various classification tasks. Each algorithm has its own methodology for processing and classifying data based on different principles, such as distance measurement, hyperplane optimization, decision rules, ensemble learning, and neural network structures.

1.3 DISADVANTAGES OF EXISTING SYSTEM

- **Extended Procedure:** Polysomnography requires an overnight stay in a sleep laboratory, which can be inconvenient for patients and disrupt their daily routines.
- **Preparation and Follow-Up:** The process involves significant preparation and follow-up visits, adding to the overall time commitment for patients.
- **Expensive Equipment and Personnel:** PSG involves costly equipment and requires specialized healthcare personnel, making it an expensive diagnostic method.
- **Long Waiting Times:** Due to the limited number of sleep centers and high demand, patients often face long waiting times for appointments.
- **Manual Analysis:** The manual scoring and interpretation of sleep data by sleep technicians and physicians can introduce human error and inconsistency in diagnoses.
- **Physical Discomfort:** The numerous sensors and electrodes attached to the body during PSG can cause physical discomfort and interfere with sleep.

1.4 PROPOSED SYSTEM

The proposed system aims to improve the classification of sleep disorders by implementing ensemble learning techniques such as the Stacking Classifier and Voting Classifier. These methods combine the strengths of multiple base models to enhance overall performance and robustness. The Stacking Classifier involves training several base models and then using another model to combine their predictions. The Voting Classifier combines the predictions of multiple models through majority voting or averaging, providing a more stable and accurate classification outcome.

1.5 ADVANTAGES OF PROPOSED SYSTEM

- **Automated Diagnosis:** The machine learning models automate the classification of sleep disorders, reducing the need for manual analysis and interpretation by sleep specialists.
- **Time-Saving:** The system provides immediate predictions, significantly reducing the time required for diagnosis compared to traditional methods.
- **Lower Costs:** The proposed system eliminates the need for expensive equipment and specialized personnel associated with polysomnography, making it a cost-effective alternative.
- **Accessibility:** The web-based application can be accessed from anywhere, reducing the need for costly overnight stays in sleep laboratories.
- **Enhanced Accuracy:** By leveraging advanced ensemble learning techniques, the system achieves high predictive accuracy, improving the reliability of diagnoses.
- **User-Friendly Interface:** The web application provides an easy-to-use platform for users to input their health data and receive predictions, making it accessible to a broad audience.
- **Remote Access:** Patients can access the diagnostic tool from the comfort of their homes, making it especially beneficial for those in remote or underserved areas.

2.ANALYSIS

2.1 INTRODUCTION

System analysis for sleep disorder classification involves the in-depth examination of the problem domain, data sources, functional requirements, and the technological framework required to build a reliable and efficient predictive model. As sleep disorders become increasingly prevalent due to modern lifestyles, stress, and health issues, accurate diagnosis has become essential for timely treatment and intervention. Traditional methods of identifying sleep-related conditions often involve expensive and time-consuming clinical procedures, such as polysomnography. However, with the advancement of digital health monitoring and machine learning, it is now possible to analyze various physiological and behavioral parameters to detect sleep disorders more effectively and efficiently.

In this system, we aim to classify different types of sleep disorders—or identify the presence of any disorder—using machine learning algorithms based on input features such as oxygen saturation, heart rate, sleep duration, snoring rate, BMI, and lifestyle indicators. The system analysis phase involves identifying the specific problems faced by healthcare providers and patients in recognizing and managing sleep issues, determining the inputs and outputs of the system, understanding the limitations of existing approaches, and outlining the key functional and non-functional requirements for an improved predictive model. This analysis helps lay a solid foundation for the system design and implementation, ensuring that the final model aligns with user needs, handles real-world data effectively, and integrates smoothly into healthcare workflows for practical use.

2.2 HARDWARE & SOFTWARE REQUIREMENTS

2.2.1 User Requirements

A PC,Mac or Laptop with x86-64 (64-bit) compatible processors

A 2 GHz or faster processor is recommended for efficient processing

A minimum of 512 MB of free RAM should be available for running the application.

An active internet connection is required for data fetching,model updates,or cloud based processing.

Microsoft Windows specific requirements:

- Operating System Supported:
 - Windows 7/8/10/11
- Additional Software:

Microsoft .NET Framework 4.5 or newer(required for .NET-based modules or GUI)
- Recommended Development Environments:

Microsoft Visual Studio 2012 or newer(for C/C++,C#,VB.NET Development)

Python IDLE or Anaconda(for Python-based ML model integration and execution)

macOS Requirements:

- Operating System Supported:
 - macOS 10.13 (High Sierra) or newer
- Development Tools:

XCode 9.3 or newer (for Swift/C/C++-based development)

GNU Make 3.81 or newer (for building code samples and system modules)

Anaconda or Python IDEs for model training and scripting

Linux Requirements:

- Kernel Version:
 - Linux kernel 3.10 or newer
- Libraries & Tools:
 - glibc 2.17 or newer
 - Gcc 4.8 or newer (for application development)
 - GNU Make 3.81 or newer (for system build and compilation)
 - Anaconda or compatible Python environment for machine learning workflow

2.2.2 Software Requirements

Operating System	: Windows 7/8/10
Server side Script	: HTML, CSS, Bootstrap & JS
Programming Language	: Python
Libraries	Flask, Pandas, Torch, Keras, Sklearn, Numpy , Seaborn
IDE/Workbench	: VSCode
Technology	: Python 3.6+
Server Deployment	: Xampp Server
Database	: MySQL

2.2.3 Hardware Requirements

Processor	- I3/Intel Processor
RAM	- 8GB (min)
Hard Disk	- 128 GB
Key Board	- Standard Windows Keyboard
Mouse	- Two or Three Button Mouse
Monitor	- Any

2.3 FLOW CHART

The Sleep Disorder Classification System operates through two interconnected workflows: the system workflow and the user workflow. The system begins by collecting sleep-related data from various sources such as sleep studies, wearable devices, or questionnaires. This data is then preprocessed to ensure it is clean and suitable for model training by handling missing values, normalizing inputs, and extracting relevant features. Following this, the system employs machine learning techniques—specifically stacking and voting classifiers—to train predictive models using base algorithms like SVM, KNN, Decision Tree, Random Forest, and ANN. Once trained, these models are evaluated using performance metrics to ensure accuracy and reliability. After validation, the model is used for real-time prediction of sleep disorders.

Simultaneously, the user interacts with the system by first registering and logging into the platform. Once authenticated, the user uploads their sleep data, which is processed by the trained model to generate a classification result indicating whether a sleep disorder is present. After viewing the result, the user can log out, ensuring the security and confidentiality of their data. This streamlined and interactive design ensures a seamless experience for users while maintaining the robustness and adaptability of the machine learning system.

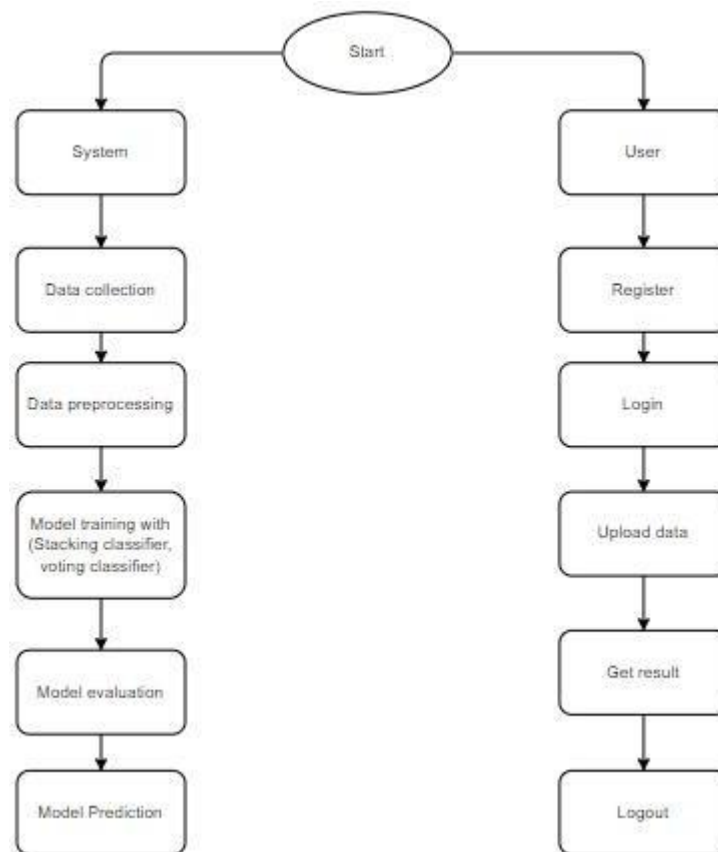


Fig 2.1 flow chart of sleep disorder classification

SYSTEM ARCHITECTURE:

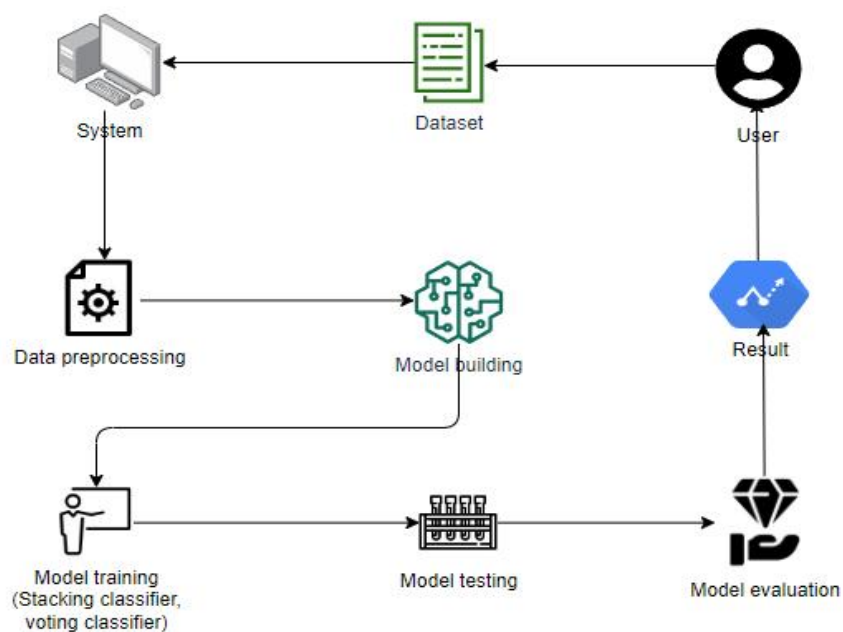


Fig 2.2 System Architecture

3.DESIGN

3.1 STEPS INVOLVED IN SLEEP DISORDER CLASSIFICATION

Entire project is divided into 3 steps as follows:

1. Data Gathering and pre processing
2. Training the model using following Machine Learning algorithms
 - i. Support Vector Machine
 - ii. K-Nearest Neighbors
 - iii. Random Forest
 - iv. Decision Trees
 - v. Artificial neural network
 - vi. Stacking Classifier
 - vii. Voting Classifier
3. Final Prediction Model integrated with frontend

3.1.1 : Data Gathering and Data Preprocessing

- a. A proper dataset is searched among various available ones and finalized with the dataset.
- b. The dataset must be pre-processed to train the model.
- c. In the preprocessing phase, the dataset is cleaned and any redundant values, noisy data and null values are removed.
- d. The pre-processed data is provided as input to the module.

3.1.2 Training the model

1. The pre-processed data is split into training and testing datasets .
2. A model is trained using the training dataset with the following algorithms
 - a. Support Vector Machine
 - b. K-Nearest Neighbors
 - c. Random Forest
 - d. Decision Trees
 - e. Artificial neural network
 - f. Stacking Classifier
 - g. Voting Classifier
3. The trained models are trained with the testing data and results are visualized using bar graphs, scatter plots.

4. The accuracy rates of each algorithm are calculated using different params like F1 score, Precision, Recall. The results are then displayed using various data visualization tools for analysis purpose.
5. The algorithm which has provided the better accuracy rate compared to remaining algorithms is taken as final prediction model.

3.1.3 : Final Sleep disorder classification

- a. The algorithm which has provided better accuracy rate has considered as the final prediction model.
- b. The model thus made is integrated with front end.
- c. Database is connected to the front end to store the user information who are using it.
- d. The output is printed on the front end as follows:
 - i. If the result is 0 the output is given as “No sleep Disorder”.
 - ii. Otherwise, the output is displayed as “Sleep Disorder”.

3.2 MODULES INVOLVED IN SLEEP DISORDER

For implementing the phishing website detection system, we have system module and user module.

3.2.1 System Module:

The system module consists of the following activities:

- Data Formatting
- Data Preprocessing
- Feature Extraction
- Model Building
- Error Handling
- Presentation Format
- Classification Results

1. Data Formatting

Data formatting is the initial stage in building a machine learning pipeline, and its primary goal is to convert raw data into a structured, clean format that can be processed by machine learning models. In many cases, data can come in various forms, such as CSV files, databases, APIs, or even unstructured sources like text, images, or audio. The first step is to

parse the raw data into structured formats, such as tables (rows and columns) or matrices that are compatible with the chosen machine learning algorithms. Proper formatting ensures consistency in the structure of the dataset, which is essential for downstream processes like preprocessing and feature extraction.

2. Data Preprocessing

Data preprocessing plays a pivotal role in improving the quality and usability of data. Raw data, often collected from real-world sources, can be noisy, incomplete, and inconsistent, which can severely affect the performance of machine learning algorithms. The first task in preprocessing is to handle missing data. There are several strategies for dealing with missing values, including imputing the missing values using the mean, median, or mode of the data, or even using more complex methods like K-nearest neighbors (KNN) imputation or interpolation. In some cases, it may be necessary to simply remove rows or columns with excessive missing data, particularly if they lack significant information. Another critical aspect of data preprocessing is feature scaling and normalization. Machine learning algorithms, especially those based on distance metrics (like K-nearest neighbors or support vector machines), are sensitive to the scale of the data. Features with larger ranges might dominate the learning process unless they are normalized to a common scale.

3. Feature Extraction

Feature extraction is a critical step in transforming raw data into meaningful representations that can be effectively used by machine learning algorithms. This step involves identifying and selecting the most important variables (or "features") from raw data, which will help the model understand patterns and make predictions. In the case of structured data, feature extraction could involve creating new features that summarize existing ones, such as computing the average or sum of certain values. For example, in a dataset of e-commerce transactions, you might create features like "average transaction amount per user" or "number of transactions in the past month" from the raw data. In domains like image and text processing, feature extraction is especially important. For text data, features can be extracted by methods such as TF-IDF (Term Frequency-Inverse Document Frequency) or word embeddings like Word2Vec. For images, features might involve extracting low-level visual patterns such as edges, textures, or color histograms, which are often required for downstream tasks like classification or object detection.

4. Model Building

Model building is the stage where machine learning algorithms are selected, trained, and optimized. The choice of algorithm depends on the type of problem (classification, regression, clustering, etc.), the nature of the data, and the computational resources available. For example, if you're dealing with a classification task, you might choose from algorithms like Logistic Regression, Support Vector Machines (SVM), or Random Forests, while for regression tasks, algorithms like Linear Regression or Decision Trees might be more suitable. The goal in this stage is to teach the algorithm to find patterns in the data that map input features to the desired output. Once the algorithm is chosen, the next task is to train the model using the preprocessed data. This involves feeding the data into the algorithm and allowing it to learn the relationships between input features and output labels. During training, the algorithm adjusts its parameters to minimize the error or loss function, using optimization techniques such as Gradient Descent.

5. Error Handling:

Error handling is a crucial component of the system that ensures the entire machine learning pipeline is robust and resilient to unexpected issues. Machine learning models and data processing pipelines often encounter errors during training or deployment due to inconsistencies in the data, misconfigured settings, or even runtime failures. During the data preprocessing stage, for instance, errors might arise when data is malformed, or invalid values are encountered. In such cases, the system should be able to detect these issues, log them for debugging, and take corrective actions such as imputing missing values, skipping corrupted records, or alerting the user to investigate the problem. During model training, errors may arise due to non-convergence (e.g., the optimization algorithm failing to find an optimal solution), overfitting (where the model performs well on training data but poorly on unseen data), or computational issues (like running out of memory). These errors should be handled gracefully, ensuring that the system can either retry the process, adjust the parameters, or provide a helpful error message to the user.

6. Presentation Format:

Once the model has been built and evaluated, the results need to be presented in a clear and understandable way for stakeholders or end-users. Presentation format involves the visual

representation of the model's predictions, performance metrics, and key insights derived from the analysis. For example, accuracy, precision, recall, and F1 score are some common metrics for classification problems, and these can be displayed in summary tables or charts. Visualizations like confusion matrices or ROC curves are often used to communicate the model's performance in a more intuitive way, especially when dealing with imbalanced datasets or complex decision boundaries.

In addition to presenting metrics, the results may also include visual dashboards or reports that allow users to interact with the system. For instance, a user might input new data into the model and receive real-time predictions, which could be displayed on a dashboard. For larger-scale applications, reports generated from the results may include explanations of model decisions, which can help users trust and understand the predictions. Tools like Tableau, Power BI, or custom web applications can be used to create these interactive reports, enabling end-users to explore model outcomes and make informed decisions.

7 Classification Results:

In classification tasks, the final output is typically a set of predicted labels, and the goal is to evaluate how well the model can classify unseen data into predefined categories. The classification results often include the predicted labels along with the associated confidence scores (i.e., the probability that the input belongs to a particular class). These confidence scores can be particularly useful when there is uncertainty, as they provide additional context about how certain the model is about its prediction. For instance, if a classifier predicts a label but provides a confidence score of only 60%, this indicates a relatively low level of certainty.

To evaluate the performance of a classification model, several metrics are used. Accuracy is the most straightforward measure, but it may not be sufficient, especially in cases where the classes are imbalanced (e.g., in fraud detection, where fraud cases are much fewer than non-fraud cases). In such cases, metrics like precision, recall, and F1 score become more important. Precision measures how many of the predicted positive instances are actually correct, while recall measures how many actual positives the model was able to identify.

3.3 UML DIAGRAM:

3.3.1 DFD Diagram

A Data Flow Diagram (DFD) is a traditional way to visualize the information flows within a system. A neat and clear DFD can depict a good amount of the system requirements graphically. It can be manual, automated, or a combination of both. It shows how information enters and leaves the system, what changes the information and where information is stored. The purpose of a DFD is to show the scope and boundaries of a system as a whole. It may be used as a communications tool between a systems analyst and any person who plays a part in the system that acts as the starting point for redesigning a system.

DFD Level-0 Diagram

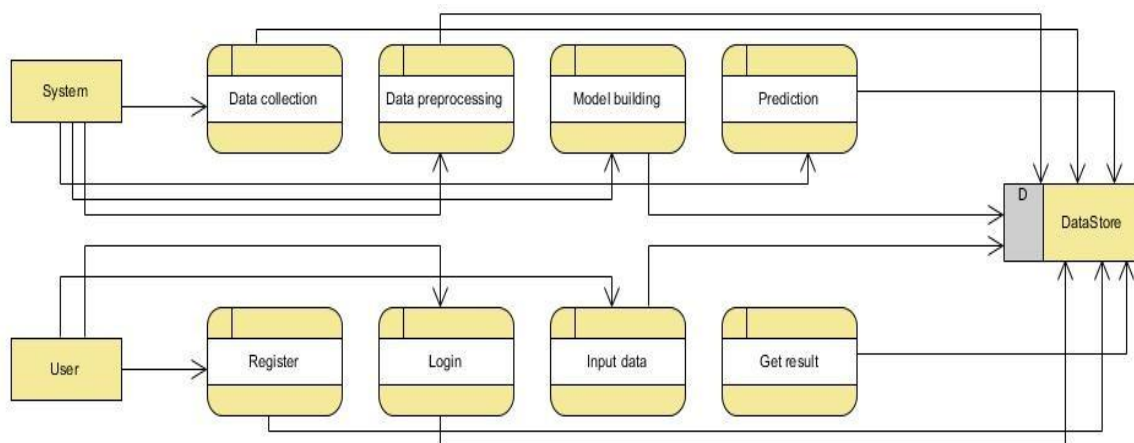


Fig 3.1 DFD Diagram Level-0

DFD Level-1 Diagram

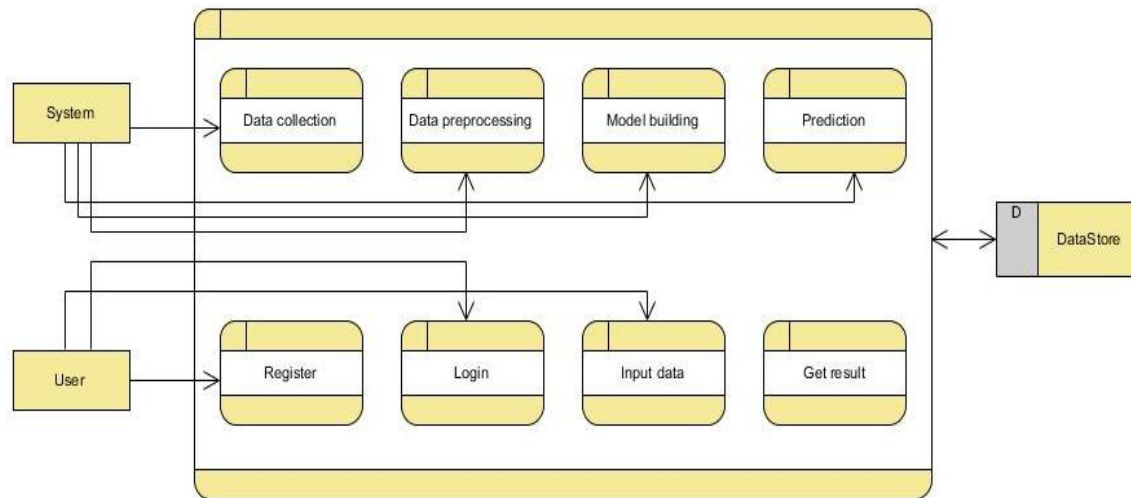


Fig 3.2 DFD Diagram Level-1

3.3.3 Class Diagram

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

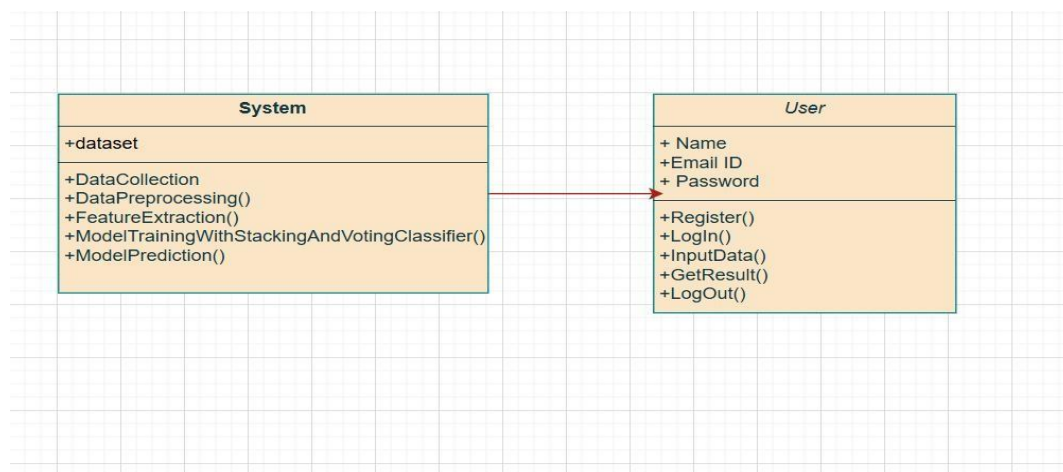


Fig 3.3 Class Diagram

3.3.4 Use Case Diagram

A Use Case Diagram (UCD) in Unified Modeling Language (UML) is a behavioral diagram that helps visualize the functional requirements of a system. It shows how the system will interact with external entities, called actors, and what actions or goals those actors aim to achieve through the system, represented as use cases. These diagrams are used primarily during the early stages of system development to capture the system's high-level functionalities and provide a user-centered view of the system.

The main purpose of a use case diagram is to identify the functions that the system will provide and to show how users (or other systems) will interact with these functions. The use cases illustrate what the system will do, not how it will do it. This distinction makes use case diagrams especially useful for gathering and documenting system requirements and communicating them to stakeholders.

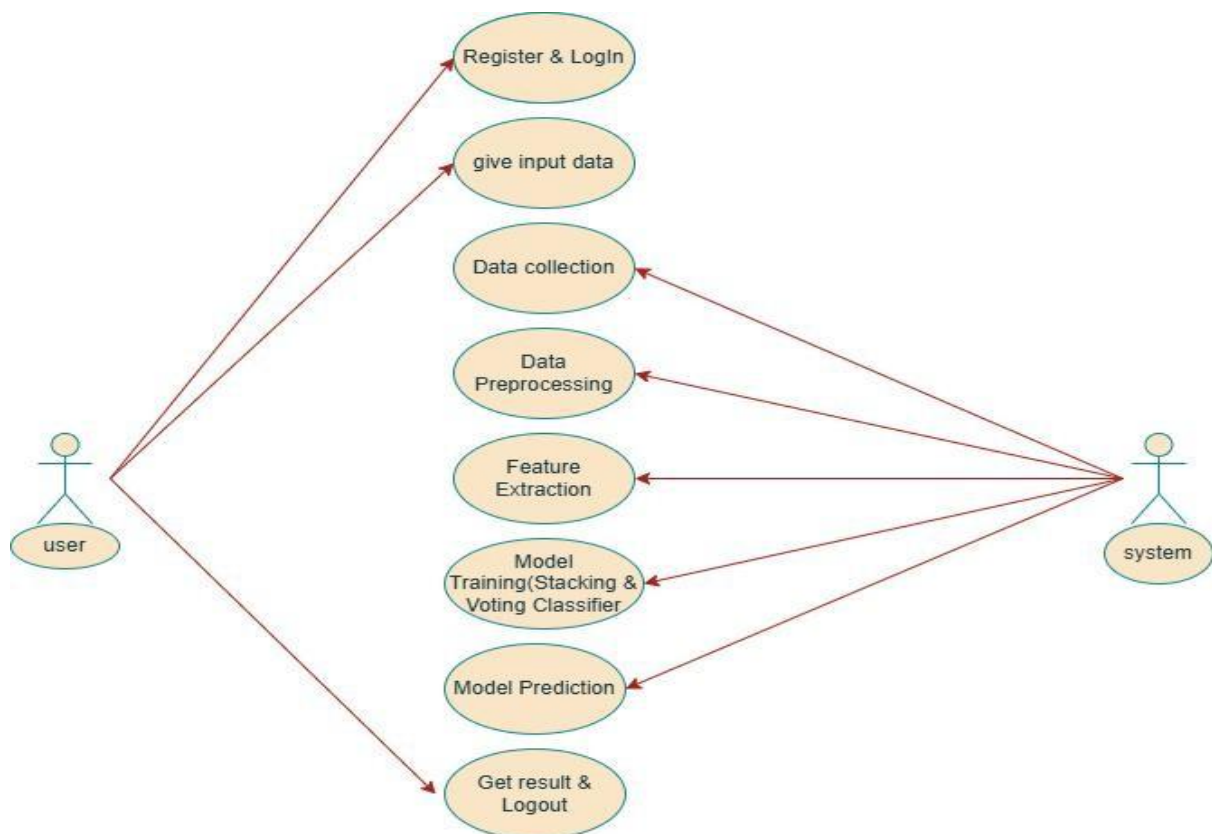


Fig 3.4 Use Case Diagram

3.3.5 Sequence Diagram

A Sequence Diagram is a type of interaction diagram in Unified Modeling Language (UML) that illustrates how objects or components in a system communicate with each other over time. It represents the sequential flow of messages between different entities (actors or objects) in a system and in what order those interactions take place. Sequence diagrams are often referred to as event diagrams, event scenarios, or timing diagrams due to their focus on events and the time sequence in which they occur.

In a sequence diagram, time is represented along the vertical axis, and objects or actors are represented along the horizontal axis. The interactions between objects are depicted as messages that are exchanged in a step-by-step manner. Each message represents a call or communication between objects in response to a specific action or request.

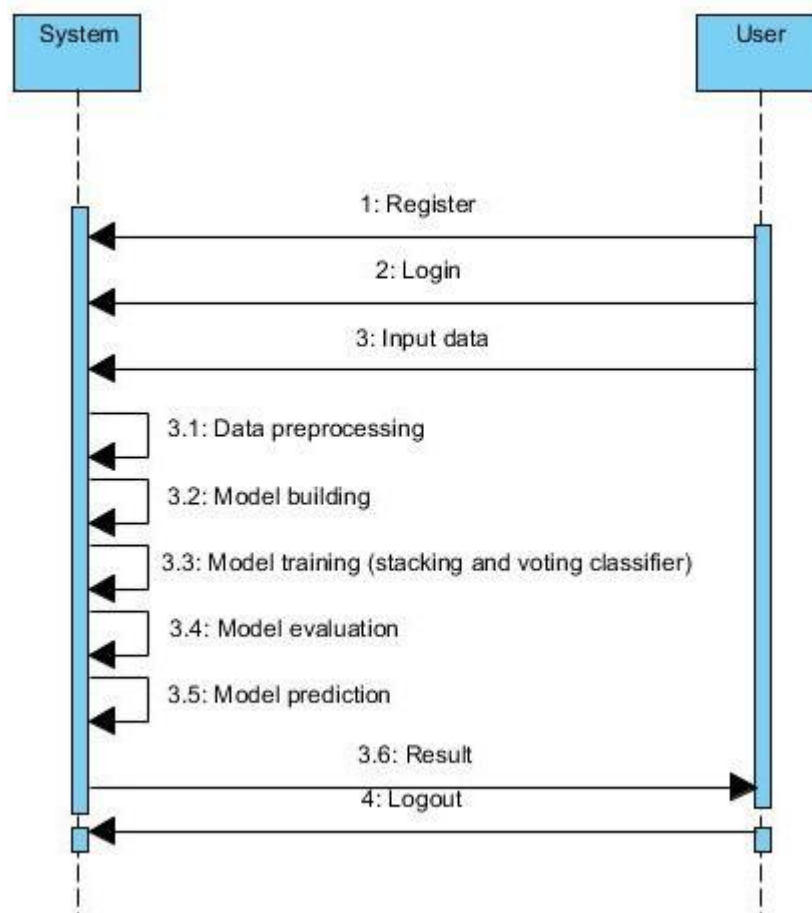


Fig 3.5 Sequence Diagram

3.3.6 Deployment Diagram

A Deployment Diagram represents the deployment view of a system. It is a type of UML diagram that illustrates the physical aspects of a system by showing how its software components are distributed across various hardware nodes (such as servers, machines, or devices). This diagram is closely related to the component diagram, as the components defined in the component diagram are deployed using deployment diagrams. The deployment diagram, therefore, focuses on the physical infrastructure, while the component diagram focuses on the logical or structural components of the system.

Deployment diagrams provide an essential representation of a system's physical architecture in UML. These diagrams not only allow stakeholders to visualize the physical deployment of software but also help in planning, scaling, and maintaining complex systems. Let's delve deeper into the important aspects of deployment diagrams, their applications, and real-world usage.



Fig 3.6 Deployment Diagram

3.3.7 Activity Diagram

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.

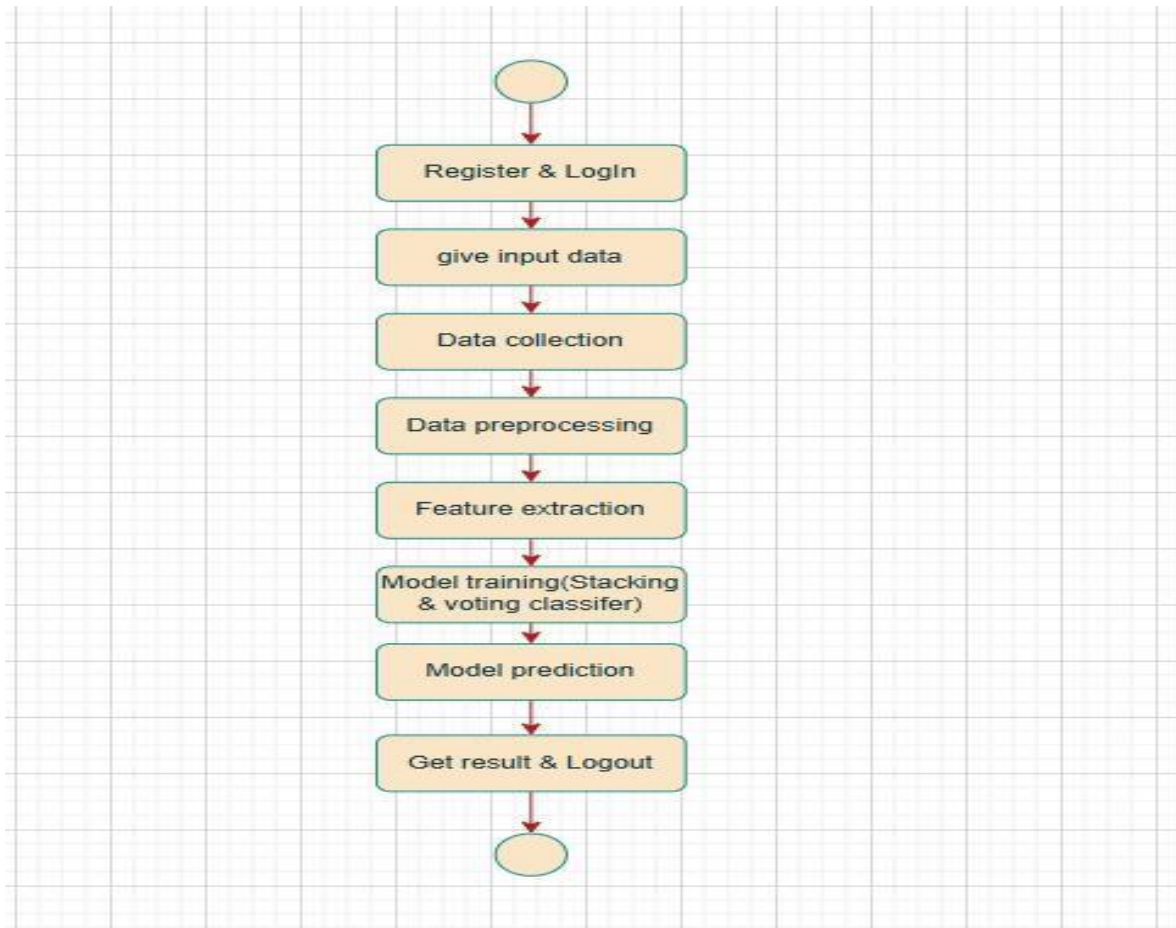


Fig 3.7 Activity Diagram

3.3.8 Collaboration Diagram

A Collaboration Diagram (also known as an Interaction Diagram or Communication Diagram) in the Unified Modeling Language (UML) is used to represent the interactions between objects or components in a system. It focuses on how objects collaborate to achieve a specific goal or functionality. The collaboration diagram shows the flow of messages between objects or components in the system and the sequence of interactions required to carry out a particular operation or use case.

While sequence diagrams emphasize the order of interactions over time, collaboration diagrams emphasize the organization of objects and their relationships. It is particularly useful for understanding how objects communicate and collaborate within the system. Collaboration diagrams provide insight into the static relationships and message exchanges between components, making them valuable for analyzing the system's behavior from a communication perspective.

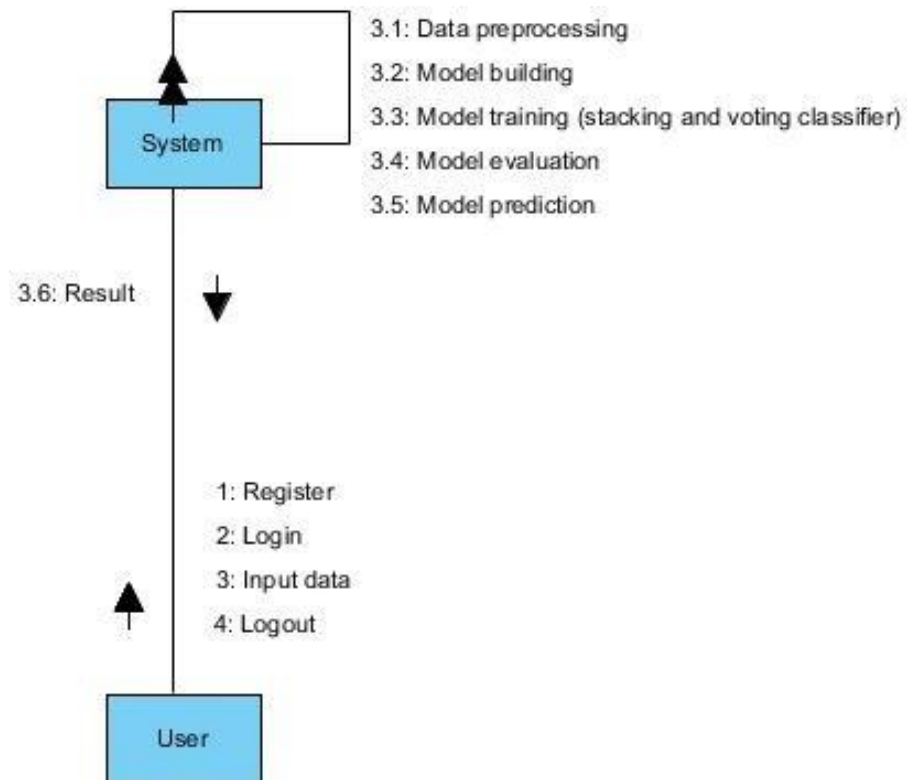


Fig 3.8 Collaboration Diagram

3.3.9 Component Diagram

A Component Diagram is one of the structural diagrams in the Unified Modeling Language (UML). It is used to model the high-level components that make up a system, their interrelationships, and their dependencies. A component diagram depicts the logical structure of a system and provides insight into how various components (such as software modules or classes) are assembled and interact to fulfill the system's functionality.

The primary goal of a component diagram is to illustrate the modularity of a system and how different components are connected. It shows how these components communicate with each other and how they fit together to create the whole system. These diagrams are useful in both design and documentation phases of software development.



Fig 3.9 Component Diagram

3.3.10 ER Model

An Entity–relationship model (ER model) describes the structure of a database with the help of a diagram, which is known as Entity Relationship Diagram (ER Diagram).

An ER diagram shows the relationship among entity sets. An entity set is a group of similar entities and these entities can have attributes.

In terms of DBMS, an entity is a table or attribute of a table in database, so by showing relationship among tables and their attributes, ER diagram shows the complete logical structure of a database.

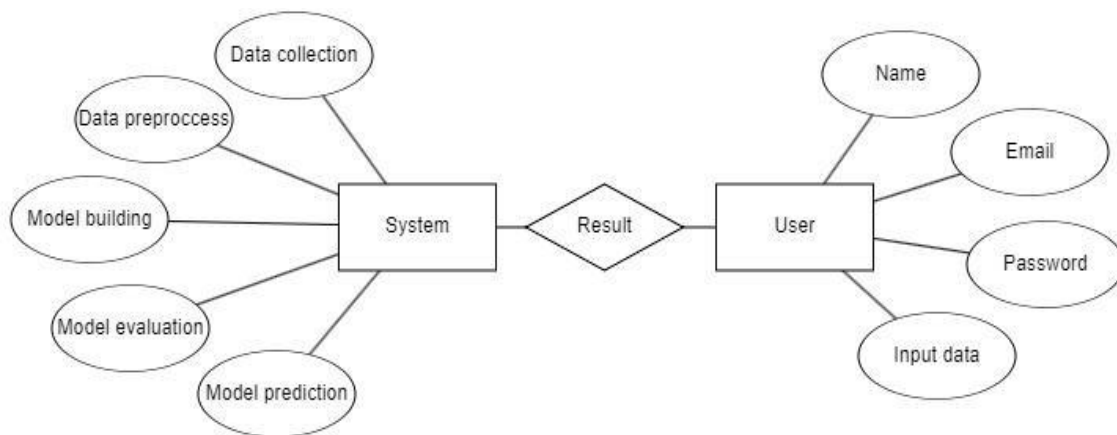


Fig 3.10 ER Model

3.3.11 Context Diagram

The given diagram represents a high-level overview of a machine learning-based system designed for the classification of sleep disorders. It is a context-level diagram that outlines how the system interacts with two main external entities: the System and the User. The central process, titled "Applying Machine Learning Algorithms for the Classification of Sleep Disorders," is responsible for analyzing data collected from both these entities to identify and classify various types of sleep disorders. The System refers to external sources such as medical databases, wearable devices, IoT-based sleep monitors, or clinical instruments like EEG and ECG, which automatically supply physiological and behavioral data to the machine learning model. On the other hand, the User provides subjective data such as sleep history, self-reported symptoms, and feedback through sleep diaries or questionnaires. This user data complements the system data and helps in personalizing the analysis. The machine learning model processes this combined data using algorithms like Random Forest, SVM, or Neural Networks to classify disorders such as insomnia, sleep apnea, narcolepsy, and more. The bidirectional flow between the model and the user also indicates that users may receive classification results, recommendations, or even interact with the system further based on the outcome. Overall, this diagram demonstrates how machine learning can effectively integrate diverse data sources to aid in the accurate and automated classification of sleep disorders.

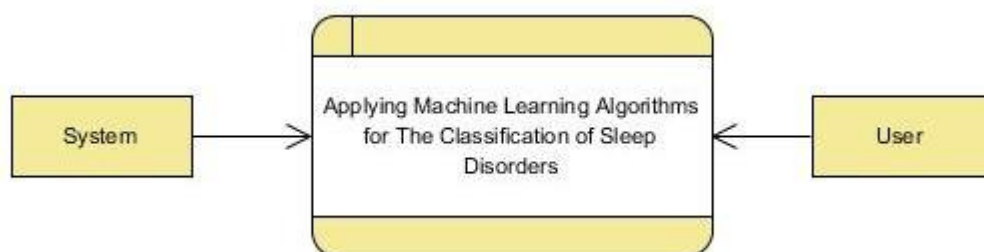


Fig 3.11 Context Diagram

4. IMPLEMENTATION AND DETAILS

4.1 INTRODUCTION

Sleep disorders have become increasingly common in modern society, affecting millions of people worldwide and significantly impacting their physical, mental, and emotional well-being. Conditions such as insomnia, sleep apnea, hypersomnia, and narcolepsy can lead to chronic fatigue, reduced productivity, cardiovascular issues, and other serious health complications if left undiagnosed or untreated. Traditional diagnostic methods like clinical interviews and polysomnography, though effective, are time-consuming, costly, and require specialized infrastructure, making early and large-scale diagnosis challenging. In response to these limitations, machine learning has emerged as a powerful tool for automating and improving the classification of sleep disorders using patient health data.

By analyzing key features such as age, gender, body mass index (BMI), stress levels, sleep duration, heart rate, snoring patterns, and lifestyle habits, machine learning algorithms can identify hidden patterns and predict the presence of specific sleep disorders with high accuracy. This study focuses on implementing and evaluating various supervised learning algorithms including Support Vector Machine (SVM), K-Nearest Neighbors (KNN), Artificial Neural Networks (ANN), Decision Tree (DT), Random Forest (RF), as well as ensemble methods like Stacking Classifier and Voting Classifier. Each of these models brings unique strengths—SVM excels at separating data in high-dimensional space, KNN is effective for simple pattern recognition, ANN mimics human neural processes, DT and RF offer interpretability and robustness, while ensemble models combine multiple learners to enhance overall performance.

The classification of sleep disorders using machine learning not only enhances diagnostic accuracy but also offers scalability, speed, and affordability, which are crucial for addressing the increasing burden of sleep health issues. Moreover, with the integration of multiple algorithms such as decision trees, random forests, support vector machines, artificial neural networks, and ensemble methods like stacking and voting classifiers, the predictive performance can be further improved. This study focuses on developing and evaluating such models to demonstrate the effectiveness of machine learning in sleep disorder classification, aiming to contribute to more accessible and proactive healthcare solutions.

Through systematic training, validation, and performance evaluation of these models on a labeled dataset, the objective is to build a reliable and scalable system capable of accurately classifying sleep disorders. The integration of such machine learning-driven systems into healthcare can assist medical professionals in early detection, reduce dependency on resource-intensive tests, and ultimately improve patient outcomes through timely intervention and personalized care.

4.1.1 Process of Sleep Disorder Classification

The classification of sleep disorders using machine learning typically involves the following steps:

1. **Data Collection:** Data collection where a comprehensive set of physiological, lifestyle, and behavioral data is gathered. Data collected from wearable devices, clinical tests such as polysomnography, surveys, and medical records provide a rich variety of features that can help differentiate between various sleep disorders, such as insomnia, sleep apnea, and narcolepsy. Commonly collected data includes sleep duration, snoring patterns, heart rate variability, BMI, daily activity levels, stress levels, and medical history. The diversity and quality of this data form the foundation for the success of the entire classification model.

2. **Data Preprocessing and Cleaning:** where raw data is prepared for analysis. Real-world datasets often come with missing values, outliers, and noise that could negatively affect model performance. Therefore, preprocessing techniques like imputation for missing data, outlier removal, and normalization of physiological features are employed. Additionally, categorical data (such as gender and medical history) needs to be encoded into numerical values, while noise filtering techniques are used to clean up sensor data. These preprocessing steps ensure that the data is of high quality and ready for further analysis.

3. **Feature Selection and Extraction:** In this phase, the most relevant features that are highly correlated with the sleep disorders are identified. Techniques like correlation analysis, recursive feature elimination (RFE), and statistical tests help in selecting important features while removing irrelevant ones. Dimensionality reduction methods like Principal Component Analysis (PCA) are also applied to simplify the data by reducing the number of features while retaining the most critical information. Effective feature selection improves model performance, reduces overfitting, and ensures the model focuses on the most important patterns in the data.

4. Model Selection and Training: Model selection and training where various supervised machine learning algorithms are implemented and compared. In this project, algorithms like Support Vector Machine (SVM), K-Nearest Neighbors (KNN), Artificial Neural Networks (ANN), Decision Tree (DT), and Random Forest (RF) are used. Additionally, ensemble methods such as Stacking Classifier and Voting Classifier are applied to leverage the strengths of multiple base learners for enhanced predictive power. These models are trained using a training subset of the data and then validated using separate testing data to evaluate generalization.

5. Model Evaluation: where the performance of each model is assessed using various evaluation metrics. Key metrics include accuracy, precision, recall, and the F1-score, which provide a clear understanding of how well the model can distinguish between different sleep disorder categories. A confusion matrix is also used to analyze the performance of the model in detail by showing the number of true positives, false positives, true negatives, and false negatives. These evaluation metrics help identify the strengths and weaknesses of each model and provide guidance for further improvement.

6. Deployment and Integration: The deployment and integration model is used to usable system. The goal here is to integrate the model into a usable system for real-time or near-real-time classification of sleep disorders. Such systems could be incorporated into health-monitoring apps, wearable devices, or clinical decision support tools. These systems provide healthcare providers with automated screening tools, enabling quicker detection of sleep disorders and facilitating timely interventions. Real-time data collection, processing, and classification make it possible to detect sleep disorders as they occur, helping patients manage their conditions more effectively. The deployment stage involves making the model accessible to end-users and integrating it with existing healthcare systems to improve patient outcomes.

4.1.2 Sleep Disorder Classification Tasks:

Sleep disorder classification using machine learning involves a series of essential tasks that work together to build a predictive system capable of identifying sleep-related health conditions with accuracy and efficiency.

1. Data Acquisition: It takes the where relevant data is collected from reliable sources such as public health datasets, clinical repositories, or wearable devices. This data typically includes attributes such as age, BMI, gender, stress level, sleep duration, heart rate, snoring intensity, and lifestyle factors like smoking and alcohol consumption. The quality and diversity of the collected data play a vital role in ensuring the model's ability to generalize across different populations.

2. Data Preprocessing and Cleaning: It takes all the raw data for analysis. This includes handling missing values, eliminating duplicates, normalizing numerical data, encoding categorical variables, and detecting outliers. Effective preprocessing ensures that the machine learning models are trained on clean, consistent, and meaningful inputs, which enhances both accuracy and model stability.

3. Feature Selection and Engineering: The feature selection are carried out to identify and construct the most relevant attributes for sleep disorder classification. Features that exhibit strong correlations with sleep disorders are selected using statistical techniques, domain knowledge, and algorithms such as Recursive Feature Elimination (RFE) or Principal Component Analysis (PCA). In some cases, new features may be engineered by combining existing ones, such as calculating sleep efficiency from sleep duration and wake time.

4. Model Selection and Implementation: It uses Multiple machine learning algorithms such as Support Vector Machine (SVM), K-Nearest Neighbors (KNN), Artificial Neural Networks (ANN), Decision Tree (DT), and Random Forest (RF) are chosen based on their strengths in classification tasks. To improve predictive performance and reduce model variance, ensemble learning techniques like Stacking Classifier and Voting Classifier are also utilized. These models are trained using a portion of the dataset and evaluated on a separate testing set to assess their generalization capabilities.

5. Training and Validation: The training task is crucial for optimizing model performance. During this phase, hyperparameter tuning is applied using techniques like grid search or random search to find the best settings for each model. Cross-validation is also used to ensure the model performs well across different subsets of data, reducing the risk of overfitting.

6. Evaluation and Performance Measurement: The evaluation task is conducted using standard metrics such as accuracy, precision, recall, F1-score, and confusion matrix. These

metrics provide insight into the model's effectiveness in correctly identifying different classes of sleep disorders and help in comparing various algorithms.

7. Deployment and Integration: The trained model can be integrated into healthcare software, mobile apps, or wearable devices to enable real-time sleep monitoring and disorder prediction. The deployment stage ensures that the benefits of the model can be utilized in practical, real-world settings for early detection and intervention.

8. System Maintenance: These are essential tasks to keep the classification system up to date. With the regular influx of new patient data and the evolution of sleep-related health patterns, the model must be retrained periodically. This ensures the system remains accurate, robust, and capable of adapting to emerging trends in sleep disorders.

4.1.3 Sleep Disorder Classification Algorithms:

The sleep disorder Classification leverages various algorithm techniques for patient sleep problems features within dataset and accurately classify them based on learned patterns. The characteristics extracted from various dataset for classification of sleep disorder. There are various algorithms used for phishing website detection. Some of them are as follows:

1. support vector machine(SVM)
2. K-Nearest Neighbours(KNN)
3. Decision Tree
4. Random Forest
5. Artificial neural network(ANN)
6. Stacking Classifier
7. Voting Classifier

4.1.4 Process Design:

The process design for sleep disorder classification involves a well-defined workflow that ensures data is accurately collected, processed, and analyzed to produce reliable predictions.

1. Data Retrieval process: where the comprehensive dataset related to sleep patterns and health metrics is collected. This data may come from publicly available health datasets, clinical research, sleep tracking devices, or patient surveys. The dataset typically includes features like sleep duration, heart rate, snoring level, stress index, BMI, oxygen saturation,

age, and lifestyle habits such as alcohol use and smoking. The main goal of this stage is to gather high-quality, representative data that reflects both individuals with and without sleep disorders.

2. Preprocessing Stage: It plays a key role in cleaning and preparing the data. This includes handling missing values through techniques like imputation, removing redundant or noisy data, and transforming categorical data into numerical formats using label encoding or one-hot encoding. Furthermore, the dataset is often normalized or standardized to ensure all features are on a similar scale, which is particularly important for distance-based algorithms like K-Nearest Neighbors (KNN) and neural networks.

3. Training and Testing Sets: It is used to split data into small parts and commonly in an 80:20 or 70:30 ratio, to allow for model training and performance evaluation.

4. Feature Generation and Extraction Process: where relevant features are selected or engineered from the dataset. Not all collected data may be useful for classification; therefore, feature selection techniques such as correlation analysis, Recursive Feature Elimination (RFE), or information gain are used to identify the most important attributes. Additionally, new features can be derived from existing ones—such as sleep efficiency, calculated using total sleep time and time in bed—which may enhance the model's ability to differentiate between sleep disorders.

5. Model Training process: Multiple machine learning models are applied to the training dataset, including Support Vector Machine (SVM), K-Nearest Neighbors (KNN), Artificial Neural Network (ANN), Decision Tree (DT), and Random Forest (RF). These models are chosen for their ability to handle classification problems with both linear and non-linear relationships. To further boost predictive accuracy, ensemble models like the Stacking Classifier and Voting Classifier are used. These techniques combine the outputs of multiple models to produce a final decision, either by majority vote (hard voting) or weighted probabilities (soft voting), often outperforming individual models.

6. Hyperparameter Tuning: It is performed using techniques like grid search or randomized search to find the optimal settings for each model, such as the number of neighbors in KNN or the number of trees in a Random Forest. The trained models are then validated using the

test dataset to assess performance. Metrics like accuracy, precision, recall, F1-score, and the confusion matrix are calculated to evaluate how well the models classify sleep disorders.

7. Deployment and Monitoring: The best-performing model is integrated into a diagnostic system or application, allowing healthcare providers or users to input relevant data and receive predictions about possible sleep disorders. Continuous monitoring of model performance is crucial, especially as new data becomes available. Periodic retraining ensures that the model remains accurate and adaptive to evolving trends and variations in patient data. This design enables a comprehensive and dynamic system that supports early diagnosis, treatment planning, and improved patient outcomes in the domain of sleep health.

Sample Code:

```
from flask import Flask, url_for, redirect, render_template, request, session
import mysql.connector, os, re
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import accuracy_score, classification_report
from xgboost import XGBClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import StackingClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.ensemble import VotingClassifier
from sklearn.preprocessing import LabelEncoder
import joblib
from imblearn.over_sampling import SMOTE
import pickle
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
app = Flask(__name__)
app.secret_key = 'admin'
```

```
mydb =
    mysql.connector.connect( host="localhost",
    user="root",
    password="21AK1A3054",
    port="3306",
    database='db'
)
mycursor = mydb.cursor()
def executionquery(query,values):
    mycursor.execute(query,values)
    mydb.commit()
    return
def retrievequery1(query,values):
    mycursor.execute(query,values)
    data = mycursor.fetchall()
    return data
def retrievequery2(query):
    mycursor.execute(query)
    data = mycursor.fetchall()
    return data
@app.route('/')
def index():
    return render_template('index.html')
@app.route('/register', methods=["GET", "POST"])
def register():
    if request.method == "POST":
        email = request.form['email']
        password = request.form['password']
        c_password = request.form['c_password']
        if password == c_password:
            query = "SELECT UPPER(email) FROM users"
            email_data = retrievequery2(query)
            email_data_list = []
            for i in email_data:
```



```
        email_data_list.append(i[0])
    if email.upper() not in email_data_list:
        query = "INSERT INTO users (email, password) VALUES (%s, %s)"
        values = (email, password)
        executionquery(query, values)
        return render_template('login.html', message="Successfully Registered!")
    return render_template('register.html', message="This email ID is already exists!")
    return render_template('register.html', message="Conform password is not match!")
    return render_template('register.html')
@app.route('/login', methods=["GET", "POST"])
def login():
    if request.method == "POST":
        email = request.form['email']
        password = request.form['password']
        query = "SELECT UPPER(email) FROM users"
        email_data = retrievequery2(query)
        email_data_list = []
        for i in email_data:
            email_data_list.append(i[0])
        if email.upper() in email_data_list:
            query = "SELECT UPPER(password) FROM users WHERE email = %s"
            values = (email,)
            password_data = retrievequery1(query, values)
            if password.upper() == password_data[0][0]:
                global user_email
                user_email = email
                return render_template('home.html')
            return render_template('login.html', message= "Invalid Password!!")
        return render_template('login.html', message= "This email ID does not exist!")
    return render_template('login.html')
@app.route('/home')
def home():
    return render_template('home.html')
@app.route('/about')
```

```
def about():
    return render_template('about.html')

@app.route('/prediction', methods=["GET", "POST"])
def prediction():
    result = None
    if request.method == "POST":
        Gender = int(request.form['Gender'])
        Age = int(request.form['Age'])
        Occupation = int(request.form['Occupation'])
        Sleep_Duration = float(request.form['Sleep_Duration'])
        Quality_of_Sleep = int(request.form['Quality_of_Sleep'])
        Physical_Activity_Level = int(request.form['Physical_Activity_Level'])
        Stress_Level = int(request.form['Stress_Level'])
        BMI_Category = request.form['BMI_Category']
        systolic = int(request.form['systolic'])
        diastolic = int(request.form['diastolic'])
        Heart_Rate = int(request.form['Heart_Rate'])
        Daily_Steps = int(request.form['Daily_Steps'])
        # Concatenate Blood Pressure
        Blood_Pressure = f"{systolic}/{diastolic}"
        # Load the scaler
        with open(r'scaler.pkl', 'rb') as f:
            scaler = pickle.load(f)
        # Load the feature selector
        with open(r'k_best_selector.pkl', 'rb') as f:
            k_best = pickle.load(f)
        # Load the stacking classifier model
        model_path = r'Random Forest_model_k_best.pkl'
        with open(model_path, 'rb') as f:
            stacking_classifier = pickle.load(f)
        single_input = {
            'Gender': Gender,
            'Age': Age,
            'Occupation': Occupation,
```

```
'Sleep Duration': Sleep_Duration,
'Quality of Sleep': Quality_of_Sleep,
'Physical Activity Level': Physical_Activity_Level,
'Stress Level': Stress_Level,
'BMI Category': BMI_Category,
'Blood Pressure': Blood_Pressure,
'Heart Rate': Heart_Rate,
'Daily Steps': Daily_Steps
}
# Convert the single input to a DataFrame
input_df = pd.DataFrame([single_input])
print(input_df)

# # Encode categorical variables using the same encoding as the training data
# input_df['Gender'] = input_df['Gender'].map({'Male': 0, 'Female': 1})
# input_df['Occupation'] = pd.Categorical(input_df['Occupation']).codes
# input_df['BMI Category'] = pd.Categorical(input_df['BMI Category']).codes
input_df['Blood Pressure'] = input_df['Blood Pressure'].str.split('/').apply(lambda x:
int(x[0]))

# Reorder the DataFrame to match the order during training
columns_order = ['Gender', 'Age', 'Occupation', 'Sleep Duration', 'Quality of Sleep',
'Physical Activity Level', 'Stress Level', 'BMI Category', 'Blood Pressure', 'Heart Rate', 'Daily
Steps']

input_df = input_df[columns_order]
# Standardize the features
input_scaled = scaler.transform(input_df)
# Select K-best features
input_k_best = k_best.transform(input_scaled)
# Predict using the stacking classifier model
prediction = stacking_classifier.predict(input_k_best)
# Map prediction to class labels
class_labels = {0: 'No sleeping disorder', 1: 'Sleeping disorder'}
result = class_labels[prediction[0]]

# Load the dataset
```

```
df = pd.read_csv(r"C:\Users\dell\Desktop\data\TK174453-Applying Machine Learning
Algorithms for the Classification of SleepDisorders\CODE\BACK
END\Sleep_health_and_lifestyle_dataset.csv")
# Drop columns
columns_to_drop = ['Sleep Disorder', 'Blood Pressure']
df = df.drop(columns=columns_to_drop)
# Replace spaces in column names with underscores
df.columns = [re.sub(r'\s+', '_', col) for col in df.columns]
# Define object columns to be encoded
object_columns = df.select_dtypes(include=['object']).columns
# Store label counts before encoding
labels = {col: df[col].value_counts().to_dict() for col in object_columns}
# Initialize LabelEncoder
le = LabelEncoder()
# Encode categorical columns and store the encoded value counts
encodes = {}
for col in object_columns:
    df[col] = le.fit_transform(df[col])
    value_counts = df[col].value_counts().to_dict()
    encodes[col] = value_counts
dic = {}
for key in labels.keys():
    dic[key] = []
    for sub_key, value in labels[key].items():
        for id_key, id_value in encodes[key].items():
            if value == id_value:
                dic[key].append((sub_key, id_key))
            break
    return render_template('prediction.html', data=dic, prediction=result)
if __name__ == '__main__':
    app.run(debug = True)
```

4.2 EXPLANATION OF KEY FEATURES

Sleep disorder is a disease to find the patient sleep problems and classify the patient sleep disease stages by using machine learning algorithms. Some key features where sleep disorder is essential include:

Insomnia (Difficulty Falling or Staying Asleep): Insomnia is one of the most common and recognizable symptoms of a sleep disorder. It refers to the persistent difficulty in falling asleep, staying asleep, or waking up too early and not being able to fall back asleep. This condition can significantly impact a person's daily life, affecting mood, energy levels, concentration, and overall health.

Excessive Daytime Sleepiness (EDS): EDS is the inability to stay awake and alert during the day. It's a hallmark of several sleep disorders like narcolepsy, sleep apnea, and insomnia. People with EDS feel an overwhelming urge to sleep, even in inappropriate situations like during meetings or driving.

Loud Snoring and Choking Sounds During Sleep: This is a common sign of obstructive sleep apnea (OSA), where the airway becomes partially or fully blocked during sleep. It leads to loud snoring, choking, or gasping for air, which briefly wakes the person up (often without them realizing).

Abnormal Movements During Sleep: This includes actions like sleepwalking, night terrors, or kicking during sleep (often in REM sleep behavior disorder or restless leg syndrome). These movements can disturb both the person and their partner.

Irregular Sleep-Wake Cycles: Also known as circadian rhythm disorders, these involve disrupted internal body clocks. People may sleep during the day and stay awake at night, or have trouble syncing sleep with normal social or work schedules.

Data Integration from Multiple Sources: The system integrates data from multiple sources such as wearable devices, smartphones, medical records, and sleep study reports to offer a comprehensive understanding of a patient's sleep patterns and health conditions. This data diversity allows the system to analyze a wide array of physiological and behavioral features, improving the accuracy and reliability of the classification process.

Real-Time Sleep Monitoring: With the help of wearable devices like smartwatches or sleep trackers, the system can monitor sleep patterns in real time. This real-time capability helps detect issues as they occur, providing immediate feedback and early alerts for potential sleep disorders. Continuous monitoring also allows for adaptive and personalized recommendations based on ongoing data.

4.3 METHOD OF IMPLEMENTATION

To implement this system, we should use machine learning algorithms such as Support Vector Machine(SVM), K-Nearest Neighbors, Random Forest, Decision Tree, Artificial Neural Network(ANN), Voting Classifier, Stacking Classifier. Machine learning, a subset of artificial intelligence, empowers computers to learn from data and improve their performance without explicit programming. This dynamic field encompasses a diverse range of techniques, algorithms, and models that enable systems to recognize patterns, make predictions, and adapt to changing scenarios. As it continues to evolve, machine learning holds the promise of revolutionizing industries, solving complex problems, and providing innovative solutions that can reshape the way we interact with technology and information. Supervised machine learning involves an input variable x and output variable y . The algorithm learns from a training dataset. With each correct answer, algorithms iteratively make predictions on the data. The learning stops when the algorithm reaches an acceptable level of performance. Unsupervised machine learning has input data X and no corresponding output variables. The goal is to model the underlying structure of the data for understanding more about the data. The keywords for supervised machine learning are classification and regression. For unsupervised machine learning, the keywords are clustering and association.

4.3.1 Machine Learning Models

Various machine learning algorithms were used in sleep disorder classification using machine learning. These machine learning models are listed as follows:

K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) is a simple, non-parametric algorithm used for classification and regression tasks. It classifies a data point based on the majority class among its k -nearest neighbors, where k is a user-defined parameter. The algorithm computes the distance between the data point and all points in the training set using distance metrics such as Euclidean, Manhattan, or Minkowski. The k points that are closest to the data point are identified, and the most common class among these neighbors determines the classification result.

In this project, KNN is applied to classify sleep disorders using health and lifestyle data. First, the dataset is preprocessed by normalizing numerical features (e.g., age, sleep duration, physical activity level) and encoding categorical variables (e.g., gender, occupation). Missing values are handled through imputation or removal. The data is then split into training and testing sets. The KNN algorithm is trained on the training set, where it learns to classify

data points based on their nearest neighbors. During prediction, the algorithm calculates the distance between a new data point and all training points, identifies the k-nearest neighbors, and assigns the class based on the majority vote among these neighbors. The value of k is optimized using cross-validation to ensure the best performance. The performance of KNN is evaluated using metrics such as accuracy, precision, recall, F1-score, and confusion matrices. While KNN is easy to implement and understand, it can be computationally expensive with large datasets, and its performance can be affected by the choice of k and distance metric. In the context of this project, KNN provides a baseline for comparison with more advanced algorithms.

Support Vector Machine (SVM)

Support Vector Machine (SVM) is a powerful and versatile supervised learning algorithm used for classification and regression tasks. It works by finding the optimal hyperplane that best separates the data points of different classes in a high-dimensional space. The hyperplane is chosen to maximize the margin, which is the distance between the hyperplane and the nearest data points from each class, known as support vectors. SVM can handle linear and non-linear classification through the use of kernel functions, which transform the input data into a higher-dimensional space where it becomes linearly separable.

In this project, SVM is employed to classify sleep disorders using health and lifestyle data. The preprocessing steps involve normalizing numerical features, encoding categorical variables, and handling missing values. The preprocessed data is then divided into training and testing sets. The SVM algorithm is trained on the training set, where it learns to identify the optimal hyperplane that separates the data points of different classes. The choice of kernel function (e.g., linear, polynomial, radial basis function) is crucial and is determined through cross-validation. During prediction, SVM projects new data points into the high-dimensional space and determines their class based on the side of the hyperplane they fall on. The performance of SVM is evaluated using metrics such as accuracy, precision, recall, F1-score, and confusion matrices. SVM is effective in high-dimensional spaces and works well with clear margin separation between classes. However, it can be computationally intensive and sensitive to the choice of hyperparameters such as the regularization parameter (C) and kernel parameters. In the context of this project, SVM provides a robust method for classifying sleep disorders, particularly when the data is not linearly separable.

Decision Tree

Decision Tree is a widely used supervised learning algorithm for classification and regression tasks. It works by recursively splitting the dataset into subsets based on the most significant feature at each node, creating a tree-like model of decisions. The goal is to partition the data in such a way that the subsets are as homogenous as possible concerning the target variable. Each internal node represents a feature, each branch represents a decision rule, and each leaf node represents a class label or a continuous value.

In this project, Decision Tree is used to classify sleep disorders using health and lifestyle data. The preprocessing steps include normalizing numerical features, encoding categorical variables, and handling missing values. The preprocessed data is then split into training and testing sets. The Decision Tree algorithm is trained on the training set, where it learns to split the data based on features that best separate the classes. The splitting criterion, such as Gini impurity or information gain, is used to determine the best feature for each split. During prediction, the algorithm traverses the tree from the root to a leaf node, following the decision rules at each node, to classify new data points. The performance of the Decision Tree is evaluated using metrics such as accuracy, precision, recall, F1-score, and confusion matrices. Decision Trees are easy to interpret and visualize, making them a popular choice for understanding the decision-making process. However, they can be prone to overfitting, especially with deep trees, and may require pruning techniques to improve generalization. In the context of this project, Decision Trees provide an interpretable method for classifying sleep disorders and serve as a base model for more complex ensemble techniques.

Random Forest

Random Forest is an ensemble learning algorithm that builds multiple decision trees and merges their predictions to improve accuracy and control overfitting. Each tree is trained on a bootstrap sample of the data, and at each split, a random subset of features is considered. The final prediction is made by averaging the predictions of all trees (for regression) or by majority voting (for classification). This randomization helps in creating a diverse set of trees, which collectively produce a more robust and accurate model.

In this project, Random Forest is utilized to classify sleep disorders using health and lifestyle data. The preprocessing steps involve normalizing numerical features, encoding categorical variables, and handling missing values. The preprocessed data is split into training and testing sets. The Random Forest algorithm is trained on the training set, where multiple

decision trees are built using different bootstrap samples and feature subsets. Each tree independently classifies the data, and the final classification is determined by aggregating the predictions of all trees. The performance of the Random Forest is evaluated using metrics such as accuracy, precision, recall, F1-score, and confusion matrices. Random Forest is advantageous because it reduces the risk of overfitting and improves predictive performance by averaging multiple decision trees. It is also less sensitive to the choice of hyperparameters compared to individual decision trees. However, it can be computationally intensive and requires careful tuning of parameters such as the number of trees and the maximum depth of each tree. In the context of this project, Random Forest serves as a powerful ensemble method for classifying sleep disorders and provides a robust baseline for comparison with more advanced techniques.

Artificial Neural Network (ANN)

Artificial Neural Network (ANN) is a computational model inspired by the human brain's neural networks. It consists of layers of interconnected nodes (neurons), where each connection has an associated weight. ANNs are capable of learning complex patterns in data through a process of training, where the weights are adjusted to minimize the error between the predicted and actual outputs. ANNs can have multiple layers, including input, hidden, and output layers, with each layer applying a non-linear transformation to the input data.

In this project, ANN is employed to classify sleep disorders using health and lifestyle data. The preprocessing steps include normalizing numerical features, encoding categorical variables, and handling missing values. The preprocessed data is split into training and testing sets. The ANN architecture is designed with an input layer corresponding to the number of features, one or more hidden layers, and an output layer with a neuron for each class. The network is trained using backpropagation, where the weights are updated iteratively to minimize the loss function, typically cross-entropy for classification tasks. During prediction, the input data is fed forward through the network, and the output layer produces class probabilities. The performance of the ANN is evaluated using metrics such as accuracy, precision, recall, F1-score, and confusion matrices. ANNs are powerful in capturing complex relationships in data and can generalize well with sufficient training data. However, they require substantial computational resources and careful tuning of hyperparameters, such as the number of layers, neurons, learning rate, and regularization parameters, to avoid overfitting. In the context of this project, ANN provides a flexible and powerful model for classifying sleep disorders, capable of handling the intricacies of health and lifestyle data.

Stacking Classifier

Stacking Classifier is an ensemble learning technique that combines multiple base models to improve predictive performance. It involves training several different models on the same dataset and then using another model, called a meta-model, to combine their predictions. The meta-model learns to predict the final output based on the predictions of the base models, leveraging their individual strengths to produce a more accurate and robust result.

In this project, the Stacking Classifier is used to classify sleep disorders using health and lifestyle data. The preprocessing steps include normalizing numerical features, encoding categorical variables, and handling missing values. The preprocessed data is split into training and testing sets. Several base models, such as KNN, SVM, Decision Tree, Random Forest, and ANN, are trained on the training set. Each base model makes predictions on the training and testing sets, and these predictions are used as input features for the meta-model. The meta-model, typically a Logistic Regression or another simple model, is trained on the predictions of the base models to learn the optimal way to combine them. During prediction, new data is first processed by the base models to generate predictions, which are then combined by the meta-model to produce the final classification. The performance of the Stacking Classifier is evaluated using metrics such as accuracy, precision, recall, F1-score, and confusion matrices. Stacking Classifier offers the advantage of combining the strengths of different models.

Voting Classifier

The Voting Classifier is an ensemble learning technique that enhances predictive performance by combining multiple base models through a voting mechanism. This method involves training various models on the same dataset and aggregating their predictions to make a final classification. Voting Classifiers can use either hard voting, where the class with the most votes from base models is chosen, or soft voting, where the class with the highest average probability is selected based on the probability scores from each model. In this project, the Voting Classifier is applied to classify sleep disorders using health and lifestyle data. The process includes normalizing numerical features, encoding categorical variables, and handling missing values before splitting the data into training and testing sets. Multiple base models, such as KNN, SVM, Decision Tree, Random Forest, and ANN, are trained, and their predictions are aggregated to determine the final classification. The Voting Classifier's performance is assessed using metrics like accuracy, precision, recall, F1-score, and

confusion matrices, offering the advantage of leveraging the strengths of diverse models to achieve improved and more robust results.

4.3.2 Output Screens

Screen1:Dataset Loading

The dataset loading has the features as shown I the figure 4.3.2.1. The dataset consists of various columns such as PersonID, Gender, Age,Occupation etc. The values in the dataset have different values.

- The value '1' represents a positive class, indicating "Sleep Disorder".
- The value '0' represents a negative class, indicating "No Sleep Disorder"

[3]:

	Person ID	Gender	Age	Occupation	Sleep Duration	Quality of Sleep	Physical Activity Level	Stress Level	BMI Category	Blood Pressure	Heart Rate	Daily Steps	Sleep Disorder
0	1	Male	27	Software Engineer	6.1	6	42	6	Overweight	126/83	77	4200	NaN
1	2	Male	28	Doctor	6.2	6	60	8	Normal	125/80	75	10000	NaN
2	3	Male	28	Doctor	6.2	6	60	8	Normal	125/80	75	10000	NaN
3	4	Male	28	Sales Representative	5.9	4	30	8	Obese	140/90	85	3000	Sleep Apnea
4	5	Male	28	Sales Representative	5.9	4	30	8	Obese	140/90	85	3000	Sleep Apnea

Screen 4.1: Dataset Loading Screen

Screen 2: Columns in dataset

The dataset consists of columns as shown in fig 4.3.2.2. The list of features such as presence of perseon ID,Gender,Age,Occupation,Sleep Duration,Qualitty of Sleep, Physical Activity Level,Stress Level,BMI Category,Blood Pressure,Heart Rate etc..

```
Index(['Person ID', 'Gender', 'Age', 'Occupation', 'Sleep Duration',
      'Quality of Sleep', 'Physical Activity Level', 'Stress Level',
      'BMI Category', 'Blood Pressure', 'Heart Rate', 'Daily Steps',
      'Sleep Disorder'],
      dtype='object')
```

Screen 4.2: Columns in dataset Screen

Screen 3: Evaluation of Support Vector Machine Model

Evaluation of the model with parameters such as precision, recall, f1-score.

```
... SVM Accuracy_original: 0.9131
```

	precision	recall	f1-score	support
0	0.92	0.85	0.91	46
1	0.85	0.90	0.94	42
accuracy			0.91	88
macro avg	0.95	0.94	0.94	88
weighted avg	0.95	0.94	0.94	88

Screen 4.3: Evaluation of Support Vector Machine Model

Screen 4: Evaluation of K-Nearest Neighbors Model

Evaluation of the model with parameters such as precision, recall, f1-score

```
[40]
... Model Performance without Feature Selection:
KNN Accuracy_original: 0.9205
```

	precision	recall	f1-score	support
0	0.91	0.93	0.92	46
1	0.93	0.90	0.92	42
accuracy			0.92	88
macro avg	0.92	0.92	0.92	88
weighted avg	0.92	0.92	0.92	88

Screen 4.4: Evaluation of K-Nearest Neighbors Model

Screen 5: Evaluation of Decision Tree Model

Evaluation of the model with parameters such as precision, recall, f1-score

```
... Decision Tree Accuracy_original: 0.9091
```

	precision	recall	f1-score	support
0	0.87	0.98	0.92	46
1	0.97	0.83	0.90	42
accuracy			0.91	88
macro avg	0.92	0.91	0.91	88
weighted avg	0.92	0.91	0.91	88

Screen 4.5: Evaluation of Decision Tree Model

Screen 6: Evaluation of Random Forest Model

Evaluation of the model with parameters such as precision, recall, f1-score

```
... Random Forest Accuracy_original: 0.8932
```

	precision	recall	f1-score	support
0	0.92	0.89	0.92	46
1	0.88	0.92	0.93	42
accuracy			0.89	88
macro avg	0.95	0.94	0.94	88
weighted avg	0.95	0.94	0.94	88

Screen 4.6: Evaluation of Random Forest Model

Screen 7: Evaluation of Artificial neural Network Model

Evaluation of the model with parameters such as precision, recall, f1-score

```
... ANN Accuracy_original: 0.8730
```

	precision	recall	f1-score	support
0	0.92	0.91	0.93	46
1	0.93	0.87	0.94	42
accuracy			0.87	88
macro avg	0.95	0.94	0.94	88
weighted avg	0.95	0.94	0.94	88

Screen 4.7: Evaluation of Artificial neural Network Model

Screen 8: Evaluation of Stacking Classifier Model

Evaluation of the model with parameters such as precision, recall, f1-score

```
... Stacking and Voting Classifier Performance with K-best Feature Selection:
Stacking Classifier Accuracy with K-best Feature Selection: 0.9432
```

	precision	recall	f1-score	support
0	0.92	0.98	0.95	46
1	0.97	0.90	0.94	42
accuracy			0.94	88
macro avg	0.95	0.94	0.94	88
weighted avg	0.95	0.94	0.94	88

Screen 4.8: Evaluation of Stacking Classifier Model

Screen 9: Evaluation of Voting Classifier Model

Evaluation of the model with parameters such as precision, recall, f1-score

```

... Voting Classifier Accuracy with K-best Feature Selection: 0.9432
      precision    recall  f1-score   support

      0       0.92       0.98       0.95        46
      1       0.97       0.90       0.94        42

 accuracy          0.94        88
  macro avg       0.95       0.94       0.94        88
  weighted avg    0.95       0.94       0.94        88
  
```

Screen 4.9: Evaluation of Voting Classifier Model

4.3.3 Result Analysis

We conducted a series of experiments to illustrate the system performance by using different machine learning models. The result shows the performance metrics for various machine learning models such as Support Vector Machine, decision tree, random forest, Artificial neural network, K-Nearest Neighbors, Stacking Classifier, Voting Classifier.

Algorithms	Accuracy	F1-Score		Recall		Precision		Support	
		0	1	0	1	0	1	0	1
Support Vector Machine	0.91	0.91	0.94	0.85	0.90	0.92	0.85	46	42
K-Nearest Neighbours	0.92	0.93	0.92	0.93	0.91	0.91	0.93	46	42
Decision Tree	0.91	0.90	0.90	0.92	0.83	0.87	0.87	46	42
Random Forest	0.89	0.92	0.93	0.89	0.92	0.92	0.88	46	42
Artificial Neural Network	0.87	0.93	0.94	0.91	0.87	0.92	0.93	46	42
Stacking Classifier	0.94	0.95	0.96	0.95	0.94	0.93	0.97	46	42
Voting Classifier	0.94	0.95	0.97	0.96	0.95	0.97	0.96	46	42

Fig:Result Analysis

5. TESTING AND VALIDATION

5.1 INTRODUCTION

Software testing is a critical element of software quality assurance and represents the ultimate review of specification, design and coding. It is the major quality measure employed during software development. Testing is the exposure of the system to trial input to see whether it produces correct output. It is a process, which reveals errors in the program. During testing, the program is executed with a set of test cases and the output of the program for the test cases is evaluated to determine if the program is performing as it is expected to perform.

5.1.1 Testing Phases:

Software testing phases include the following:

Unit Testing:

It aims to check whether each individual piece of the code, like a function or a method, works correctly on its own. It focuses on the internal logic of the code, making sure that each unit gives the expected output for a given input. The main goal is to catch and fix bugs early in the development process, even before different parts of the software are combined.

This phase is usually performed by developers during the coding phase, often before the integration of different modules begins. By doing so, it ensures early detection of bugs, making them easier and cheaper to fix. Developers typically write automated unit tests using frameworks such as JUnit for Java, NUnit for .NET applications, or pytest for Python. These tools help execute multiple test cases quickly and support test-driven development (TDD), where tests are written before the actual code. Unit testing promotes code quality, improves maintainability, and serves as valuable documentation for the expected behavior of individual components.

Integration Testing:

It is done to make sure that different modules or components of the software work together as expected. Once unit testing confirms that individual parts work, integration testing checks whether the connections between those parts, such as data passing or function calls, are

correct and smooth. It helps in detecting any issues in the way different units interact with each other.

This phase is usually conducted by developers and QA testers, depending on the development strategy. It can be done using different approaches like top-down integration, bottom-up integration, or big bang integration. Tools such as Postman (for API testing), TestNG, and SOAP UI are commonly used to validate data exchange and integration points, especially in service-oriented applications. Integration testing plays a critical role in distributed systems, microservices, and layered architecture, where proper communication between components is essential.

System Testing:

It takes the fully developed software and checks if the entire system functions properly. It verifies that the software meets all the defined requirements and works well as a complete product. This level of testing simulates real-world scenarios to see if the software behaves the way users expect, including testing its performance, security, and overall reliability.

System testing is typically performed by professional QA testers who use predefined test plans and test cases to assess the software's quality. It includes various subtypes such as functional testing, performance testing, usability testing, compatibility testing, and security testing. Tools like Selenium (for automated functional testing), LoadRunner and JMeter (for performance testing), and TestComplete are frequently used. The insights gained from system testing help stakeholders evaluate the system's readiness for acceptance testing and eventual deployment.

Acceptance Testing:

It is the final stage, where the software is tested from the end-user or client's perspective. The goal here is to confirm that the software fulfills the business requirements and is ready for release. It is often done in a real environment or by actual users to ensure that the product works as intended in practical usage conditions.

This phase may be carried out by the client, product owner, or a select group of end-users with support from the QA team. It typically involves two types of testing: Alpha Testing, which is done internally before release, and Beta Testing, which is done externally by real

users. Acceptance testing ensures the product's usability, reliability, and overall satisfaction from a user's point of view. Tools like Cucumber and FitNesse are commonly used in behavior-driven development (BDD) environments, where acceptance criteria are written in plain language to bridge the gap between technical and non-technical stakeholders.

5.1.2 Testing Methods:

Testing is a process of executing a program to find out errors. If testing is conducted successfully, it will uncover all the errors in the software. Any testing can be done basing on two ways:

Testing is a systematic process of executing a program with the intent of identifying and correcting errors. When conducted effectively, testing can reveal both obvious and hidden defects, ensuring that the software meets its intended functionality and quality standards. There are two primary methods of software testing based on the level of information available to the tester: White Box Testing and Black Box Testing.

White Box Testing

White Box Testing, also known as clear box testing, structural testing, or glass box testing, is a method where the tester has full access to the internal workings of the software. This testing technique involves validating the internal logic, code structure, and system architecture. The tester uses their knowledge of the code to create test cases that examine various paths, branches, and conditions within the software, ensuring that every line of code is executed at least once. White box testing focuses on verifying that the software's internal components function as expected, identifying logical errors, redundant code, and potential security vulnerabilities.

The key advantage of white box testing is that it allows for a deep analysis of the software's inner mechanics, often uncovering defects that are not apparent from an external perspective. Developers usually perform this testing during the unit testing phase, ensuring that individual functions or methods are working as intended before they are integrated into the larger system. Techniques such as statement coverage, branch coverage, and path coverage are commonly used to measure the extent of testing and ensure thorough validation. Tools like JUnit, NUnit, and SonarQube are often employed to automate and support white box testing processes, helping ensure that the codebase maintains a high level of quality.

Black Box Testing

Black Box Testing is a testing method where the tester has no knowledge of the internal structure or logic of the software being tested. Instead, the tester interacts with the software as an end-user would, providing inputs and examining the outputs. The focus of black box testing is to validate the functional behavior of the system, ensuring that it meets the specified requirements and user expectations. Test cases are typically designed based on the software's functional specifications, such as user stories or requirements documents, and they assess how well the application performs its tasks.

Black box testing is particularly effective for validating the software from the end-user's perspective. It helps ensure that the software behaves correctly under normal, edge, and error conditions without requiring knowledge of the code. Since this method tests the software's external behavior, it is commonly used during system testing, integration testing, and acceptance testing. Testers perform operations like input validation, error handling, and interface testing to ensure the system functions as intended. Tools such as Selenium, QTP (UFT), and Appium are often used to automate black box testing, making it easier to simulate user interactions and validate the application's responses.

5.1.3 Testing Approach:

Software testing approaches define how the testing process is conducted based on the sequence in which components or modules are tested. The two primary testing approaches are Bottom-Up Approach and Top-Down Approach. Both strategies focus on integrating and validating different modules of the system, but they follow different sequences and methods for executing tests.

Testing can be done in two ways:

- Bottom-up approach
- Top-down approach

Bottom-up Approach:

Bottom-Up Approach:

testing begins from the lowest-level modules and gradually progresses upward to the higher-level modules. This approach is often used in integration testing, where the smaller, foundational components are tested before integrating them into the larger system. The idea is

to validate the functionality of the smallest building blocks first, ensuring that each unit of the system behaves as expected. Once the lower-level modules pass their individual tests, they are integrated into higher-level modules, which are tested one at a time. During this process, drivers are used as temporary substitutes for higher-level modules that are not yet ready. Drivers simulate the behavior of those modules to test the lower-level components effectively.

The primary advantage of the Bottom-Up Approach is that it allows testers to ensure the basic functionality of individual modules before integrating them with other parts of the system. This makes it easier to isolate issues and fix them at an early stage. However, one downside is that testing the complete system behavior may be delayed until the higher-level modules are integrated. Tools and techniques for Bottom-Up testing include unit tests and automated tests, often executed using frameworks such as JUnit or NUnit. Overall, this approach is particularly effective in applications where lower-level logic is crucial, such as embedded systems or applications with complex data handling.

Top-Down Approach:

The Top-Down Approach, in contrast, begins testing with the highest-level modules and moves downward to the lower-level components. The process starts with testing the main control module or the primary business logic of the system. Since lower-level modules may not be available for integration at the beginning, stubs are used. A stub is a small, dummy module that mimics the behavior of the real module by returning predefined values to simulate a proper interaction between the modules. This approach allows the tester to focus on verifying the control flow and interactions between major system components early in the testing process.

The Top-Down Approach is useful for quickly validating the high-level functionality and ensuring that the primary system functions as expected. However, a major limitation is that the lower-level modules, which may be critical to the overall functionality, are not tested immediately. This could lead to delays in detecting issues that arise from these lower-level components. This approach is commonly employed when the design and overall system functionality are a priority early in development, such as in large-scale enterprise applications. Once the stubs are replaced with real modules, the testing continues downward to ensure that every module works as intended. Testing tools and techniques for this approach often include stubs, integration tests, and high-level system validation tools.

5.2 DESIGN OF EVALUATION MATRICES

Algorithms	Accuracy	F1-Score		Recall		Precision		Support	
		0	1	0	1	0	1	0	1
Support Vector Machine	91.31%	91%	94%	85%	90%	92%	85%	46	42
K-Nearest Neighbours	92.05%	93%	92%	93%	91%	91%	93%	46	42
Decision Tree	90.91%	90%	90%	92%	83%	87%	87%	46	42
Random Forest	89.32%	92%	93%	89%	92%	92%	88%	46	42
Artificial Neural Network	87.30%	93%	94%	91%	87%	92%	93%	46	42
Stacking Classifier	94.32%	95%	96%	95%	94%	93%	97%	46	42
Voting Classifier	94.28%	95%	97%	96%	95%	97%	96%	46	42

5.3 VALIDATION

The system has been tested and implemented successfully and thus ensured that all the requirements as listed in the software requirements specification are completely fulfilled. In case of erroneous input corresponding error messages are displayed.

6.CONCLUSION & FUTURE ENHANCEMENT

6.1 CONCLUSION

This project effectively demonstrates the application of machine learning algorithms for the classification of sleep disorders using health and lifestyle data. By employing ensemble learning techniques like Stacking and Voting Classifiers, the system not only enhances the accuracy and robustness of the classification process but also outperforms traditional diagnostic methods. The use of multiple machine learning algorithms allows for addressing the limitations inherent in individual models, leading to more reliable and efficient classification of sleep disorders. These methods help mitigate issues such as overfitting, underfitting, and bias, making the system more adaptable to different types of sleep disorder data.

Throughout the project, the data was successfully preprocessed and formatted to ensure it was suitable for machine learning model training and prediction. This step is crucial for ensuring that the models can learn meaningful patterns from the data without being influenced by noise or inconsistencies. By carefully cleaning the data, applying normalization, and handling missing values, the system becomes capable of producing actionable insights that can aid healthcare providers in identifying sleep disorders with greater accuracy. The results indicate that the approach significantly improves diagnostic accessibility and cost-effectiveness, especially when compared to traditional diagnostic methods, which often require costly and time-consuming procedures.

One of the major advantages of this approach is its scalability. The system is designed to be adaptable to a wide range of data sources, including wearables, electronic health records (EHR), and other patient monitoring devices. This flexibility makes it applicable across various healthcare settings, such as hospitals, sleep clinics, and home healthcare systems. Additionally, the use of machine learning enables the system to handle large datasets, which may not be possible with traditional diagnostic tools. This scalability opens up opportunities for the system to be used in diverse real-world applications, from clinical environments to consumer health products.

6.2 FUTURE ENHANCEMENT

Future enhancements to the sleep disorder classification project present several exciting opportunities for improving both the accuracy and accessibility of the system. One of the most promising areas for improvement is the integration of advanced deep learning models such as Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTM) networks. These models can be particularly effective in processing complex and sequential data, such as EEG signals or heart rate variability, which play a crucial role in diagnosing sleep disorders. CNNs excel in extracting hierarchical features from raw data, which is valuable for analyzing signal patterns in physiological data. On the other hand, LSTMs are highly suited for time-series analysis, making them ideal for understanding the temporal patterns in sleep data, such as sleep cycles, heart rate fluctuations, and disruptions during sleep. Incorporating these advanced models would significantly improve the system's ability to detect subtle, non-linear patterns in the data, resulting in more accurate and reliable classifications of sleep disorders.

Another critical area for future enhancement is the implementation of real-time data processing and prediction capabilities. Real-time processing would allow the system to detect and diagnose sleep disorders as they occur, providing immediate feedback to users and healthcare providers. For instance, if a wearable device detects irregularities in a user's heart rate or oxygen levels during sleep, the system could immediately flag these anomalies and suggest further action. Real-time prediction capabilities could be particularly valuable for individuals with chronic sleep issues, as it would enable proactive management of their condition, allowing for timely interventions. Additionally, real-time monitoring could help users track improvements or changes in their sleep patterns over time, contributing to a more dynamic and personalized approach to sleep disorder management.

7. BIBLIOGRAPHY

1. **Alshammari, T. S. (2024).** Applying Machine Learning Algorithms for the Classification of Sleep Disorders
2. **Arslan, R. S. (2023).** Sleep disorder and apnea events detection framework with high performance using two-tier learning model design.
3. **The research, featured in *Traitement Du Signal*,** offers an in-depth examination of multiple tree-based algorithms, emphasizing their efficiency in precisely classifying various sleep stages.
4. **Chatur, A., Haghi, M., Ganapathy, N., Taherinejad, N., Seepold, R., & Madrid, N. M. (2024).** Advanced Classifiers and Feature Reduction for Accurate Insomnia Detection Using Multimodal Dataset.
5. **The study by Cheng, Lech, and Wilkinson (2023).** focuses on detecting sleep stages and sleep disorders simultaneously by employing deep learning techniques with data collected from multiple sensors.
6. **Han, H., Seong, M. J., Hyeon, J., Joo, E., and Gracious, J. (2024).** investigated the arrangement and robotized assessment of excitement force across various phases of rest utilizing AI strategies. Their discoveries were distributed in Logical Reports 2024, volume 14, article number 1, pages 1-12. The review is open at <https://doi.org/10.1038/s41598-023-50653-9>.
7. **Huang, A. A., and Huang, S. Y. (2023).** Use of AI strategies to decide the gamble factors related with sleep deprivation.
8. **Jiao, M., Song, C., Xian, X., Yang, S., & Liu, F. (2024).** use of deep attention networks that integrate information from multiple time periods to improve the detection of sleep apnea
9. **Ji, X., Li, Y., and Wen, P. (2023).** A Profound Learning Approach for Grouping Rest Stages Utilizing Multi-Channel Bio-Signs: 3DSleepNet
10. **Kong, G., Li, C., Peng, H., Han, Z., and Qiao, H. (2023).** IEEE Exchanges on Brain Frameworks and Restoration Designing, Volume 31, Pages 1075-1085.
11. **Lee, M., Kang, H., Yu, S. H., Cho, H., Oh, J., Lande, G. van der, Gosseries, O., & Jeong, J. H. (2024).** Automatic Sleep Stage Classification Using Nasal Pressure Decoding Based on a Multi-Kernel Convolutional BiLSTM Network. Maniaci, A., Riela, P. M., Iannella, G., Lechien, J. R., La Mantia, I., De Vincentiis, M., Cammaroto, G., Calvo-Henriquez, C., Di Luca, M., Chiesa Estomba, C., Saibene, A. M., Pollicina, I., Stilo, G., Di Mauro, P., Cannavici, A., Lugo, R., Magliulo, G., Greco.
12. **Mirth, J. R., Felton, C. L., Haider, C. R., McCarter, S. J., Morgenthaler, T. I., Louis, E. K. S., & Holmes, D. R. (2023).** Identification of Sleep Patterns via Clustering of Hypnodensities.

- 13. Peng, C. C., & Kou, C. Y. (2023).** Sleep Disorder Classification Using Convolutional Neural Networks
- 14. Salsone, M., Vescio, B., Quattrone, A., Marelli, S., Castelnovo, A., Casoni, F., Quattrone, A., & Ferini-Strambi, L. (2024).** Periodic Leg Movements during Sleep Associated with REM Sleep Behavior Disorder
- 15. Zhang, Z., Lin, B. S., Peng, C. W., & Lin, B. S. (2024).** Multi-Modal Sleep Stage Classification With Two-Stream Encoder-Decoder.